

CT60A2600-TIIMI-A

Generated by Doxygen 1.9.1

1 Class Index	1
1.1 Class List	1
2 File Index	3
2.1 File List	3
3 Class Documentation	5
3.1 jono Struct Reference	5
3.1.1 Detailed Description	5
3.1.2 Member Data Documentation	5
3.1.2.1 pSeuraava	5
3.1.2.2 pSolmu	5
3.2 puu Struct Reference	6
3.2.1 Detailed Description	6
3.2.2 Member Data Documentation	6
3.2.2.1 aNimi	6
3.2.2.2 iArvo	6
3.2.2.3 iPituus	6
3.2.2.4 pOikea	7
3.2.2.5 pVasen	7
3.3 RBSolmu Struct Reference	7
3.3.1 Detailed Description	7
3.3.2 Member Data Documentation	7
3.3.2.1 aNimi	7
3.3.2.2 iArvo	8
3.3.2.3 iVariBitti	8
3.3.2.4 pOikea	8
3.3.2.5 pVanhempi	8
3.3.2.6 pVasen	8
3.4 tiedot Struct Reference	8
3.4.1 Detailed Description	9
3.4.2 Member Data Documentation	9
3.4.2.1 aSukunimi	9
3.4.2.2 iYhteensa	9
3.4.2.3 pEdellinen	9
3.4.2.4 pSeuraava	9
4 File Documentation	11
4.1 binaaripuu.c File Reference	11
4.1.1 Function Documentation	12
4.1.1.1 etsiPienin()	12
4.1.1.2 kirjoitaBinaaripuu()	12
4.1.1.3 kirjoitaTiedostoon()	13

4.1.1.4 lisaaSolmu()	13
4.1.1.5 luoPuu()	14
4.1.1.6 nimenArvo()	15
4.1.1.7 oikeaPuoli()	16
4.1.1.8 onkoLuku()	16
4.1.1.9 paivitaPuu()	17
4.1.1.10 poistaSolmu()	18
4.1.1.11 puunPituus()	19
4.1.1.12 suurempiLukuVertailu()	19
4.1.1.13 tasapainoitaPuu()	20
4.1.1.14 vapautaMuistiPuu()	20
4.1.1.15 varaaMuistiaPuulle()	21
4.1.1.16 vasenPuoli()	21
4.2 binaaripuu.h File Reference	22
4.2.1 Typedef Documentation	23
4.2.1.1 PUU	23
4.2.1.2 RBSOLMU	23
4.2.2 Function Documentation	24
4.2.2.1 etsiPienin()	24
4.2.2.2 kierraOikealle()	24
4.2.2.3 kierraVasemmalle()	25
4.2.2.4 kirjoitaBinaaripuu()	25
4.2.2.5 kirjoitaRB()	25
4.2.2.6 kirjoitaRBTiedostoon()	26
4.2.2.7 kirjoitaTiedostoon()	26
4.2.2.8 korjaaLisays()	27
4.2.2.9 lisaaRBSolmu()	28
4.2.2.10 lisaaSolmu()	28
4.2.2.11 luoPuu()	29
4.2.2.12 luoRBPuu()	30
4.2.2.13 nimenArvo()	31
4.2.2.14 oikeaPuoli()	31
4.2.2.15 onkoLuku()	32
4.2.2.16 paivitaPuu()	33
4.2.2.17 poistaSolmu()	33
4.2.2.18 puunPituus()	34
4.2.2.19 suurempiLukuVertailu()	35
4.2.2.20 tasapainoitaPuu()	35
4.2.2.21 vapautaMuistiPuu()	36
4.2.2.22 vapautaMuistiRB()	36
4.2.2.23 varaaMuistiaPuulle()	37
4.2.2.24 varaaMuistiaRB()	38

4.2.2.25 vasenPuoli()	38
4.3 haut.c File Reference	39
4.3.1 Function Documentation	39
4.3.1.1 binaarihaku()	40
4.3.1.2 leveyshaku()	40
4.3.1.3 syvyyshaku()	41
4.3.1.4 tarkistaLoytvykoSyvyyshaula()	42
4.3.1.5 vapautaMuistiJono()	43
4.3.1.6 varaaMuistiaJonolle()	43
4.4 haut.h File Reference	44
4.4.1 Typedef Documentation	44
4.4.1.1 JONO	44
4.4.2 Function Documentation	44
4.4.2.1 binaarihaku()	44
4.4.2.2 leveyshaku()	45
4.4.2.3 syvyyshaku()	46
4.4.2.4 tarkistaLoytvykoSyvyyshaula()	47
4.4.2.5 vapautaMuistiJono()	47
4.4.2.6 varaaMuistiaJonolle()	48
4.5 linkitettylista.c File Reference	48
4.5.1 Function Documentation	49
4.5.1.1 halkaise()	49
4.5.1.2 kirjoitaTiedostoAlustaLoppuun()	50
4.5.1.3 kirjoitaTiedostoLopustaAlkuun()	51
4.5.1.4 laskeListanPituus()	51
4.5.1.5 lisaaListaan()	52
4.5.1.6 lisaysLajittelu()	53
4.5.1.7 lomituz()	54
4.5.1.8 lomituzLajittelu()	55
4.5.1.9 lue()	56
4.5.1.10 onkoLukuVaiNimi()	57
4.5.1.11 poistaListastaAlkio()	57
4.5.1.12 poistaListastaLkmPerusteella()	58
4.5.1.13 poistaListastaNimenPerusteella()	59
4.5.1.14 useammallaAlkiollaSamaLKM()	60
4.5.1.15 vapautaMuisti()	60
4.5.1.16 varaaMuistia()	61
4.6 linkitettylista.h File Reference	61
4.6.1 Typedef Documentation	62
4.6.1.1 TIEDOT	62
4.6.2 Function Documentation	62
4.6.2.1 halkaise()	62

4.6.2.2 kirjoitaTiedostoAlustaLoppuun()	63
4.6.2.3 kirjoitaTiedostoLopustaAlkuun()	64
4.6.2.4 laskeListanPituus()	64
4.6.2.5 lisaaListaan()	65
4.6.2.6 lisaysLajittelu()	66
4.6.2.7 lomituss()	67
4.6.2.8 lomitussLajittelu()	68
4.6.2.9 lue()	69
4.6.2.10 onkoLukuVaiNimi()	70
4.6.2.11 poistaListastaAlkio()	70
4.6.2.12 poistaListastaLkmPerusteella()	71
4.6.2.13 poistaListastaNimenPerusteella()	72
4.6.2.14 useammallaAlkiollaSamaLKM()	73
4.6.2.15 vapautaMuisti()	73
4.6.2.16 varaaMuistia()	74
4.7 paaohjelma.c File Reference	74
4.7.1 Function Documentation	75
4.7.1.1 main()	75
4.8 punamustapuu.c File Reference	75
4.8.1 Function Documentation	76
4.8.1.1 kierraOikealle()	76
4.8.1.2 kierraVasemmalle()	76
4.8.1.3 kirjoitaRB()	77
4.8.1.4 kirjoitaRBTiedostoon()	77
4.8.1.5 korjaaLisays()	77
4.8.1.6 lisaaRBSolmu()	78
4.8.1.7 luoRBPuu()	79
4.8.1.8 vapautaMuistiRB()	79
4.8.1.9 varaaMuistiaRB()	80
4.9 yleiset.c File Reference	80
4.9.1 Function Documentation	80
4.9.1.1 binaaripuuValikko()	81
4.9.1.2 kysyArvo()	81
4.9.1.3 kysyNimi()	82
4.9.1.4 listaValikko()	82
4.9.1.5 mainBinaaripuu()	83
4.9.1.6 mainLista()	84
4.9.1.7 valikko()	85
4.10 yleiset.h File Reference	85
4.10.1 Macro Definition Documentation	86
4.10.1.1 LEN	86
4.10.2 Function Documentation	86

4.10.2.1 binaaripuuValikko()	86
4.10.2.2 kysyArvo()	87
4.10.2.3 kysyNimi()	87
4.10.2.4 listaValikko()	88
4.10.2.5 mainBinaaripuu()	88
4.10.2.6 mainLista()	90
4.10.2.7 valikko()	91

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

jono		5
puu		6
RBSolmu		7
tiedot		8

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

binaaripuu.c	11
binaaripuu.h	22
haut.c	39
haut.h	44
linkitettylista.c	48
linkitettylista.h	61
paaohjelma.c	74
punamustapuu.c	75
yleiset.c	80
yleiset.h	85

Chapter 3

Class Documentation

3.1 jono Struct Reference

```
#include <haut.h>
```

Public Attributes

- [PUU](#) * [pSolmu](#)
- struct [jono](#) * [pSeuraava](#)

3.1.1 Detailed Description

Definition at line 5 of file haut.h.

3.1.2 Member Data Documentation

3.1.2.1 pSeuraava

```
struct jono* jono::pSeuraava
```

Definition at line 7 of file haut.h.

3.1.2.2 pSolmu

```
PUU* jono::pSolmu
```

Definition at line 6 of file haut.h.

The documentation for this struct was generated from the following file:

- [haut.h](#)

3.2 puu Struct Reference

```
#include <binaaripuu.h>
```

Public Attributes

- char [aNimi](#) [[LEN](#)]
- int [iArvo](#)
- int [iPituus](#)
- struct [puu](#) * [pOikea](#)
- struct [puu](#) * [pVasen](#)

3.2.1 Detailed Description

Definition at line 5 of file binaaripuu.h.

3.2.2 Member Data Documentation

3.2.2.1 aNimi

```
char puu::aNimi [LEN]
```

Definition at line 6 of file binaaripuu.h.

3.2.2.2 iArvo

```
int puu::iArvo
```

Definition at line 7 of file binaaripuu.h.

3.2.2.3 iPituus

```
int puu::iPituus
```

Definition at line 8 of file binaaripuu.h.

3.2.2.4 pOikea

```
struct puu* puu::pOikea
```

Definition at line 9 of file binaaripuu.h.

3.2.2.5 pVasen

```
struct puu* puu::pVasen
```

Definition at line 10 of file binaaripuu.h.

The documentation for this struct was generated from the following file:

- [binaaripuu.h](#)

3.3 RBSolmu Struct Reference

```
#include <binaaripuu.h>
```

Public Attributes

- char [aNimi](#) [[LEN](#)]
- int [iArvo](#)
- int [iVariBitti](#)
- struct [RBSolmu](#) * [pOikea](#)
- struct [RBSolmu](#) * [pVasen](#)
- struct [RBSolmu](#) * [pVanhempi](#)

3.3.1 Detailed Description

Definition at line 13 of file binaaripuu.h.

3.3.2 Member Data Documentation

3.3.2.1 aNimi

```
char RBSolmu::aNimi [LEN]
```

Definition at line 14 of file binaaripuu.h.

3.3.2.2 iArvo

```
int RBSolmu::iArvo
```

Definition at line 15 of file binaaripuu.h.

3.3.2.3 iVariBitti

```
int RBSolmu::iVariBitti
```

Definition at line 16 of file binaaripuu.h.

3.3.2.4 pOikea

```
struct RBSolmu* RBSolmu::pOikea
```

Definition at line 17 of file binaaripuu.h.

3.3.2.5 pVanhempi

```
struct RBSolmu* RBSolmu::pVanhempi
```

Definition at line 19 of file binaaripuu.h.

3.3.2.6 pVasen

```
struct RBSolmu* RBSolmu::pVasen
```

Definition at line 18 of file binaaripuu.h.

The documentation for this struct was generated from the following file:

- [binaaripuu.h](#)

3.4 tiedot Struct Reference

```
#include <linkitettylista.h>
```


Public Attributes

- char [aSukunimi](#) [[LEN](#)]
- int [iYhteensa](#)
- struct [tiedot](#) * [pSeuraava](#)
- struct [tiedot](#) * [pEdellinen](#)

3.4.1 Detailed Description

Definition at line 5 of file linkitettylista.h.

3.4.2 Member Data Documentation

3.4.2.1 aSukunimi

```
char tiedot::aSukunimi[LEN]
```

Definition at line 6 of file linkitettylista.h.

3.4.2.2 iYhteensa

```
int tiedot::iYhteensa
```

Definition at line 7 of file linkitettylista.h.

3.4.2.3 pEdellinen

```
struct tiedot* tiedot::pEdellinen
```

Definition at line 9 of file linkitettylista.h.

3.4.2.4 pSeuraava

```
struct tiedot* tiedot::pSeuraava
```

Definition at line 8 of file linkitettylista.h.

The documentation for this struct was generated from the following file:

- [linkitettylista.h](#)

Chapter 4

File Documentation

4.1 binaaripuu.c File Reference

```
#include "binaaripuu.h"
#include <ctype.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

Functions

- [PUU * varaaMuistiaPuulle](#) (char *pSolmu, int iArvo)
Muistin varaaminen puu structin alkioille.
- [PUU * vapautaMuistiPuu](#) (PUU *pJuuriSolmu)
Muistin vapauttaminen puu structin alkioista Juurisolmun ja kaikkien sen lapsie solmujen muisti vapautetaan.
- [PUU * lisaaSolmu](#) (PUU *pAlku, char *pSolmu, int iArvo)
Lisaa solmun puuhun.
- [PUU * luoPuu](#) (char *pNimi, [PUU](#) *pJuuriSolmu)
Luetaan tiedosto ja luodaan puu rakenne.
- void [kirjoitaBinaaripuu](#) (char *pNimi, [PUU](#) *pAlku)
Tasapainoitettun binaaripuun kirjoittaminen tiedostoon.
- void [kirjoitaTiedostoon](#) (char *pNimi, [PUU](#) *pAlku)
Kirjoittaa binaaripuun tiedostoon.
- int [tasapainoitaPuu](#) (PUU *pAlku)
Palauttaa solmun tasapainokertoimen.
- [PUU * oikeaPuoli](#) (PUU *pAlku)
Oikean puolen kierto.
- [PUU * vasenPuoli](#) (PUU *pAlku)
Vasemman puolen kierto.
- int [puunPituus](#) (PUU *pAlku)
Hakee solmun korkeuden.
- int [suurempiLukuVertailu](#) (int iLuku1, int iLuku2)
Vertaa kahta lukua ja palauttaa suuremman.
- [PUU * poistaSolmu](#) (char *pNimi, [PUU](#) *pJuuriSolmu)

- *Lehtisolmun poistaminen.*
int **onkoLuku** (char *pNimi)
Testaa, onko käyttäjän antama arvo nimi vai numeroarvo.
- int **nimenArvo** (char *pNimi, **PUU** *pJuuriSolmu)
Etsii nimen perusteella sen arvon.
- **PUU** * **etsiPienin** (**PUU** *pUusiSolmu)
Etsii pienimmän solmun arvon binääripuusta.
- **PUU** * **paivitaPuu** (**PUU** *pJuuriSolmu)
Tasapainotetaan puu poiston jälkeen.

4.1.1 Function Documentation

4.1.1.1 etsiPienin()

```
PUU* etsiPienin (
    PUU * pUusiSolmu )
```

Parameters

<i>pUusiSolmu</i>	Osoitin kasiteltavaan solmuun.
-------------------	--------------------------------

Returns

PUU* Palauttaa osoittimen puun pienimpaan solmuun.

Definition at line 447 of file binaaripuu.c.

```
447     {
448     PUU *pNykyinenSolmu = pUusiSolmu;
449
450     while (pNykyinenSolmu->pVasen != NULL) {
451         pNykyinenSolmu = pNykyinenSolmu->pVasen;
452     }
453
454     return (pNykyinenSolmu);
455 }
```

4.1.1.2 kirjoitaBinaaripuu()

```
void kirjoitaBinaaripuu (
    char * pNimi,
    PUU * pAlku )
```

Parameters

<i>pNimi</i>	Kirjoitettavan tiedoston nimi.
<i>pAlku</i>	

Definition at line 176 of file binaaripuu.c.

```

176                                     {
177     /*if (pAlku == NULL) { //Noora kommentoi pois 19.3.2026 klo. 13.08
178         printf("Puu on tyhjä, luo puurakenne ennen kirjoittamista.\n");
179         return;
180     }*/
181     if (pAlku != NULL) {
182         kirjoitaTiedostoon(pNimi, pAlku);
183         kirjoitaBinaaripuu(pNimi, pAlku->pVasen);
184         kirjoitaBinaaripuu(pNimi, pAlku->pOikea);
185     }
186     return;
187 }
```

4.1.1.3 kirjoitaTiedostoon()

```

void kirjoitaTiedostoon (
    char * pNimi,
    PUU * pAlku )
```

Parameters

<i>pNimi</i>	Kirjoitettavan tiedoston nimi
<i>pAlku</i>	Solmu, joka kirjoitetaan tiedostoon

Definition at line 195 of file binaaripuu.c.

```

195                                     {
196     FILE *Tiedosto = NULL;
197
198     if (pAlku == NULL) {
199         return;
200     }
201
202     /* Tiedoston avaaminen. */
203     if ((Tiedosto = fopen(pNimi, "a")) == NULL) {
204         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
205         exit(0);
206     }
207     /* Tiedostoon kirjoittaminen. */
208     fprintf(Tiedosto, "%s,%d\n", pAlku->aNimi, pAlku->iArvo);
209
210     /* Tiedoston sulkeminen. */
211     fclose(Tiedosto);
212     return;
213 }
```

4.1.1.4 lisaaSolmu()

```

PUU* lisaaSolmu (
    PUU * pAlku,
    char * pSolmu,
    int iArvo )
```

Tasapainottaa puun.

Parameters

<i>pAlku</i>	
<i>pSolmu</i>	Lisattava solmu.
<i>iArvo</i>	Solmun arvo.

Returns

PUU*

Definition at line 58 of file binaaripuu.c.

```

58                                     {
59     int iVertailu = 0;
60
61     // Varataan muistia, jos muisti tyhjä.
62     if (pAlku == NULL) {
63         return varaaMuistiaPuulle(pSolmu, iArvo);
64     }
65
66     iVertailu = strcmp(pSolmu, pAlku->aNimi);
67
68     // Solmujen lisääminen.
69     if (iArvo < pAlku->iArvo) {
70         pAlku->pVasen = lisaaSolmu(pAlku->pVasen, pSolmu, iArvo);
71     } else if (iArvo > pAlku->iArvo) {
72         pAlku->pOikea = lisaaSolmu(pAlku->pOikea, pSolmu, iArvo);
73     }
74
75     // Testataan, ovatko arvot samat.
76     } else if (iArvo == pAlku->iArvo) {
77         if (iVertailu < 0) {
78             pAlku->pVasen = lisaaSolmu(pAlku->pVasen, pSolmu, iArvo);
79         } else if (iVertailu > 0) {
80             pAlku->pOikea = lisaaSolmu(pAlku->pOikea, pSolmu, iArvo);
81         }
82     }
83
84     // Selvitetaan paikka, johon solmu lisataan. Tasapainotetaan puu.
85     pAlku->iPituus = 1 + suurempiLukuVertailu(puunPituus(pAlku->pVasen), puunPituus(pAlku->pOikea));
86     int tasapaino = tasapainoitaPuu(pAlku);
87
88     // vasen vasen tasapainotus
89     if ((tasapaino > 1) && (iArvo < pAlku->pVasen->iArvo)) {
90         return (oikeaPuoli(pAlku));
91     }
92
93     // vasen oikea tasapainotus
94     if ((tasapaino > 1) && (iArvo > pAlku->pVasen->iArvo)) {
95         pAlku->pVasen = vasenPuoli(pAlku->pVasen);
96         return (oikeaPuoli(pAlku));
97     }
98
99     // oikea vasen tasapainotus
100    if ((tasapaino < -1) && (iArvo < pAlku->pOikea->iArvo)) {
101        pAlku->pOikea = oikeaPuoli(pAlku->pOikea);
102        return (vasenPuoli(pAlku));
103    }
104
105    // oikea oikea tasapainotus
106    if ((tasapaino < -1) && (iArvo > pAlku->pOikea->iArvo)) {
107        return (vasenPuoli(pAlku));
108    }
109    return (pAlku);
110 }

```

4.1.1.5 luoPuu()

```

PUU* luoPuu (
    char * pNimi,
    PUU * pJuuriSolmu )

```

Pilkotaan luetut rivit.

Parameters

<i>pNimi</i>	Luettavan tiedoston nimi.
<i>pJuuriSolmu</i>	Puun juurisolmu.

Returns

PUU* Palauttaa juuriSolmun.

Definition at line 119 of file binaaripuu.c.

```

119                                     {
120     FILE *Tiedosto = NULL;
121     char aRivi[LEN] = "";
122     int iOtsikko = 0;
123     char *p1 = NULL, *p2 = NULL;
124     int iArvo = 0;
125
126     // Tiedoston avaus
127     if ((Tiedosto = fopen(pNimi, "r")) == NULL) {
128         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
129         exit(0);
130     }
131
132     // Tiedoston lukeminen
133     while ((fgets(aRivi, LEN, Tiedosto)) != NULL) {
134         aRivi[strlen(aRivi) - 1] = '\0';
135
136         // Ohitetaan otsikko
137         if (iOtsikko == 0) {
138             iOtsikko++;
139             continue;
140         }
141
142         // Pilkkotaan rivit
143         if ((p1 = strtok(aRivi, ";")) == NULL) {
144             perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
145             exit(0);
146         }
147
148         if ((p2 = strtok(NULL, ";")) == NULL) {
149             perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
150             exit(0);
151         }
152
153         iArvo = atoi(p2);
154
155         // Maaritetaan juuri solmu jos sitä ei ole määritetty.
156         if (pJuuriSolmu == NULL) {
157             pJuuriSolmu = lisaSolmu(pJuuriSolmu, p1, iArvo);
158             iOtsikko++;
159             continue;
160         }
161
162         pJuuriSolmu = lisaSolmu(pJuuriSolmu, p1, iArvo);
163     }
164
165     // Suljetaan tiedosto
166     fclose(Tiedosto);
167     return (pJuuriSolmu);
168 }

```

4.1.1.6 nimenArvo()

```

int nimenArvo (
    char * pNimi,
    PUU * pJuuriSolmu )

```

Parameters

<i>pNimi</i>	Poistettava nimi merkkijonona.
<i>pJuuriSolmu</i>	Osoitin kasiteltavaan solmuun.

Returns

int Palauttaa 0 tai 1, riippuen siitä, löytyykö nimen nimen pohjalta arvoa.

Definition at line 422 of file binaaripuu.c.

```

422                                     {
423     if (pJuuriSolmu == NULL) {
424         return (0);
425     }
426
427     if (strcmp(pJuuriSolmu->aNimi, pNimi) == 0) {
428         int iArvo = pJuuriSolmu->iArvo;
429         return (iArvo);
430     }
431
432     int iArvo = nimenArvo(pNimi, pJuuriSolmu->pVasen);
433     if (iArvo != 0) {
434         return (iArvo);
435     } else {
436         int iArvo = nimenArvo(pNimi, pJuuriSolmu->pOikea);
437         return (iArvo);
438     }
439 }

```

4.1.1.7 oikeaPuoli()

```

PUU* oikeaPuoli (
    PUU * pAlku )

```

Puu järjestetään, jotta se on tasapainossa.

Parameters

<i>pAlku</i>	
--------------	--

Returns

PUU* Palauttaa solmun.

Definition at line 237 of file binaaripuu.c.

```

237                                     {
238     // Tarkistetaan, onko pAlku tai pAlku oikea puoli NULL
239     if ((pAlku->pVasen == NULL) || (pAlku == NULL)) {
240         return (pAlku);
241     }
242
243     // Muuttujien ja vakioiden alustaminen
244     PUU *pMuuttujal = pAlku->pVasen;
245     PUU *pMuuttuja2 = pMuuttujal->pOikea;
246
247     // Sijoitetaan arvoja
248     pMuuttujal->pOikea = pAlku;
249     pAlku->pVasen = pMuuttuja2;
250
251     // Verrataan ja sijoitetaan suuremmat korkeudet
252     pAlku->iPituus = suurempiLukuVertailu(puunPituus(pAlku->pVasen), puunPituus(pAlku->pOikea)) + 1;
253     pMuuttujal->iPituus =
254         suurempiLukuVertailu(puunPituus(pMuuttujal->pVasen), puunPituus(pMuuttujal->pOikea)) + 1;
255
256     return pMuuttujal;
257 }

```

4.1.1.8 onkoLuku()

```

int onkoLuku (
    char * pNimi )

```

Ilmoittaa, onko käyttäjän antama numero vai ei.

Parameters

<i>pNimi</i>	Poistettava nimi tai numeroarvo merkkijonona.
--------------	---

Returns

int Palauttaa 0 tai 1, riippuen siitä onko käyttäjän syöte numero.

Definition at line 401 of file binaaripuu.c.

```
401     {
402         int tosi = 0;
403         char *ptr = pNimi;
404
405         while (*ptr) {
406             if (!isdigit(*ptr)) {
407                 return tosi;
408             }
409             ptr++;
410         }
411         tosi = 1;
412         return (tos);
413     }
```

4.1.1.9 paivitaPuu()

```
PUU* paivitaPuu (
    PUU * pJuuriSolmu )
```

Parameters

<i>pJuuriSolmu</i>	Osoitin kasiteltavaan solmuun.
--------------------	--------------------------------

Returns

PUU* palauttaa tasapainotetun solmun.

Definition at line 463 of file binaaripuu.c.

```
463     {
464         // Puun korkeuden päivittäminen.
465         if (pJuuriSolmu == NULL) {
466             return (pJuuriSolmu);
467         }
468
469         int iLuku1 = puunPituus(pJuuriSolmu->pVasen);
470         int iLuku2 = puunPituus(pJuuriSolmu->pOikea);
471         pJuuriSolmu->iPituus = suurempiLukuVertailu(iLuku1, iLuku2) + 1;
472
473         // Puun tasapanottaminen.
474         int iTasapaino = tasapainoitaPuu(pJuuriSolmu);
475         // Vasen-vasen tapaus.
476         if (iTasapaino > 1 && tasapainoitaPuu(pJuuriSolmu->pVasen) >= 0) {
477             return oikeaPuoli(pJuuriSolmu);
478         }
479
480         // Oikea-oikea tapaus.
481         if (iTasapaino < -1 && tasapainoitaPuu(pJuuriSolmu->pOikea) <= 0) {
482             return vasenPuoli(pJuuriSolmu);
483         }
484
485         // Vasen-oikea tapaus.
486         if (iTasapaino > 1 && tasapainoitaPuu(pJuuriSolmu->pVasen) < 0) {
487             pJuuriSolmu->pVasen = vasenPuoli(pJuuriSolmu->pVasen);
488             return oikeaPuoli(pJuuriSolmu);
489         }
```

```

489     }
490
491     // Oikea-vasen tapaus.
492     if (iTasapaino < -1 && tasapainoitaPuu(pJuuriSolmu->pOikea) > 0) {
493         pJuuriSolmu->pOikea = oikeaPuoli(pJuuriSolmu->pOikea);
494         return vasenPuoli(pJuuriSolmu);
495     }
496
497     return (pJuuriSolmu);
498 }

```

4.1.1.10 poistaSolmu()

```

PUU* poistaSolmu (
    char * pNimi,
    PUU * pJuuriSolmu )

```

Poistaa lehtisolmun joko numeroarvon tai nimen perusteella.

Parameters

<i>pNimi</i>	Poistettava nimi tai numeroarvo merkkijonona.
<i>pJuuriSolmu</i>	Osoitin kasiteltavaan puun solmuun.
<i>iArvo</i>	Haettava numeroarvo.

Returns

PUU* Uusi osoitin puun solmuun, NULL jos puu tyhjeni.

Definition at line 334 of file binaaripuu.c.

```

334     {
335         int iArvo = 0;
336         PUU *pUusiSolmu = NULL;
337
338         if (pJuuriSolmu == NULL) {
339             printf("Solmua ei löytynyt.\n");
340             return (pJuuriSolmu);
341         }
342
343         // Testataan, onko syöte nimi vai lukuarvo, ja määritetään lukuarvo puun läpikäymiseksi.
344         if (onkoLuku(pNimi)) {
345             iArvo = atoi(pNimi);
346         } else {
347             iArvo = nimenArvo(pNimi, pJuuriSolmu);
348
349             if (iArvo == 0) {
350                 printf("Solmua ei löytynyt tällä nimellä.\n");
351                 return (pJuuriSolmu);
352             }
353         }
354
355         // Lehtisolmun poistaminen.
356         if (iArvo < pJuuriSolmu->iArvo) {
357             pJuuriSolmu->pVasen = poistaSolmu(pNimi, pJuuriSolmu->pVasen);
358         } else if (iArvo > pJuuriSolmu->iArvo) {
359             pJuuriSolmu->pOikea = poistaSolmu(pNimi, pJuuriSolmu->pOikea);
360         } else {
361
362             if (pJuuriSolmu->pVasen == NULL && pJuuriSolmu->pOikea == NULL) {
363                 free(pJuuriSolmu);
364                 pJuuriSolmu = NULL;
365                 printf("Solmu poistettu.\n");
366                 return (pJuuriSolmu);
367             } else if (pJuuriSolmu->pVasen == NULL || pJuuriSolmu->pOikea == NULL) {
368                 if (pJuuriSolmu->pVasen != NULL) {
369                     pUusiSolmu = pJuuriSolmu->pVasen;
370                 } else {

```

```

372         pUusiSolmu = pJuuriSolmu->pOikea;
373     }
374     free(pJuuriSolmu);
375     pJuuriSolmu = NULL;
376     printf("Solmu poistettu.\n");
377     return (pUusiSolmu);
378 }
379 } else {
380     pUusiSolmu = etsiPienin(pJuuriSolmu->pOikea);
381     pJuuriSolmu->iArvo = pUusiSolmu->iArvo;
382     strcpy(pJuuriSolmu->aNimi, pUusiSolmu->aNimi);
383     pJuuriSolmu->pOikea = poistaSolmu(pUusiSolmu->aNimi, pJuuriSolmu->pOikea);
384     return (pJuuriSolmu);
385 }
386 }
387
388 pJuuriSolmu = paivitaPuu(pJuuriSolmu);
389 return (pJuuriSolmu);
390 }

```

4.1.1.11 puunPituus()

```

int puunPituus (
    PUU * pAlku )

```

Parameters

<i>pAlku</i>	Kohta, josta korkeus haetaan
--------------	------------------------------

Returns

int Palauttaa solmun korkeuden

Definition at line 293 of file binaaripuu.c.

```

293     {
294         // Tarkistetaan, onko Solmu Null
295         if (pAlku == NULL) {
296             return (0);
297         }
298
299         // Palauttaa solmun korkeuden
300         return (pAlku->iPituus);
301     }

```

4.1.1.12 suurempiLukuVertailu()

```

int suurempiLukuVertailu (
    int iLuku1,
    int iLuku2 )

```

Parameters

<i>iLuku1</i>	Ensimmäinen verrattava kokonaisluku.
<i>iLuku2</i>	Toinen verrattava kokonaisluku.

Returns

int Palauttaa suuremman luvun.

Definition at line 310 of file binaaripuu.c.

```
310                                     {
311     int iPalautus = 0;
312
313     // Verrataan kumpi luvuista on suurempi.
314     if (iLuku1 > iLuku2) {
315         iPalautus = iLuku1;
316     } else {
317         iPalautus = iLuku2;
318     }
319
320     // Palauttaa suuremman luvun.
321     return (iPalautus);
322 }
```

4.1.1.13 tasapainoitaPuu()

```
int tasapainoitaPuu (
    PUU * pAlku )
```

Parameters

<i>pAlku</i>	Solmu, josta tasapainokerroin halutaan
--------------	--

Returns

int

Definition at line 221 of file binaaripuu.c.

```
221                                     {
222     // Tarkistetaan, onko Null
223     if (pAlku == NULL) {
224         return (0);
225     }
226
227     // Palautetaan tasapainokerroin
228     return (puunPituus(pAlku->pVasen) - puunPituus(pAlku->pOikea));
229 }
```

4.1.1.14 vapautaMuistiPuu()

```
PUU* vapautaMuistiPuu (
    PUU * pJuuriSolmu )
```

Parameters

<i>pJuuriSolmu</i>	Puun juurisolmu
--------------------	-----------------

Returns

PUU*

Definition at line 39 of file binaaripuu.c.

```
39                                     {
40     // Muistin vapauttaminen
41     if (pJuuriSolmu != NULL) {
42         vapautaMuistiPuu(pJuuriSolmu->pVasen);
43         vapautaMuistiPuu(pJuuriSolmu->pOikea);
44         free(pJuuriSolmu);
45     }
46     pJuuriSolmu = NULL;
47     return (pJuuriSolmu);
48 }
```

4.1.1.15 varaaMuistiaPuulle()

```
PUU* varaaMuistiaPuulle (
    char * pSolmu,
    int iArvo )
```

Parameters

<i>pSolmu</i>	Solmu, jolle muistia varataan
<i>iArvo</i>	Arvo, jolle muistia varataan

Returns

PUU*

Definition at line 16 of file binaaripuu.c.

```
16                                     {
17     PUU *pUusi = NULL;
18
19     // Muistin varaaminen
20     if ((pUusi = (PUU *)malloc(sizeof(PUU))) == NULL) {
21         perror("Muistin varaus epäonnistui, lopetetaan");
22         exit(0);
23     }
24
25     strcpy(pUusi->aNimi, pSolmu);
26     pUusi->iArvo = iArvo;
27     pUusi->iPituus = 1;
28     pUusi->pVasen = NULL;
29     pUusi->pOikea = NULL;
30     return (pUusi);
31 }
```

4.1.1.16 vasenPuoli()

```
PUU* vasenPuoli (
    PUU * pAlku )
```

Puu järjestetään, jotta se on tasapainossa.

Parameters

<i>pAlku</i>	
--------------	--

Returns

PUU* Palauttaa solmun.

Definition at line 265 of file binaaripuu.c.

```

265     {
266         // Tarkistetaan, onko pAlku tai pAlku oikea puoli NULL
267         if ((pAlku->pOikea == NULL) || (pAlku == NULL)) {
268             return (pAlku);
269         }
270
271         // Muuttujien ja vakioiden alustaminen
272         PUU *pMuuttuja1 = pAlku->pOikea;
273         PUU *pMuuttuja2 = pMuuttuja1->pVasen;
274
275         // Sijoitetetaan arvoja
276         pMuuttuja1->pVasen = pAlku;
277         pAlku->pOikea = pMuuttuja2;
278
279         // Verrataan ja sijoitetaan suuremmat korkeudet
280         pAlku->iPituus = suurempiLukuVertailu(puunPituus(pAlku->pVasen), puunPituus(pAlku->pOikea)) + 1;
281         pMuuttuja1->iPituus =
282             suurempiLukuVertailu(puunPituus(pMuuttuja1->pVasen), puunPituus(pMuuttuja1->pOikea)) + 1;
283
284         return pMuuttuja1;
285     }

```

4.2 binaaripuu.h File Reference

```
#include "yleiset.h"
```

Classes

- struct [puu](#)
- struct [RBSolmu](#)

Typedefs

- typedef struct [puu](#) PUU
- typedef struct [RBSolmu](#) RBSOLMU

Functions

- [PUU * varaaMuistiaPuulle](#) (char *pSolmu, int iArvo)
Muistin varaaminen puu structin alkioille.
- [PUU * vapautaMuistiPuu](#) (PUU *pJuuriSolmu)
Muistin vapauttaminen puu structin alkioista Juurisolmun ja kaikkien sen lapsie solmujen muisti vapautetaan.
- [PUU * lisaaSolmu](#) (PUU *pAlku, char *pSolmu, int iArvo)
Lisaa solmun puuhun.
- [PUU * luoPuu](#) (char *pNimi, [PUU](#) *pJuuriSolmu)

- Luetaan tiedosto ja luodaan puu rakenne.*
- int `tasapainoitaPuu` (`PUU *pAlku`)
- Palauttaa solmun tasapainokertoimen.*
- `PUU * oikeaPuoli` (`PUU *pAlku`)
- Oikean puolen kierto.*
- `PUU * vasenPuoli` (`PUU *pAlku`)
- Vasemman puolen kierto.*
- int `puunPituus` (`PUU *pAlku`)
- Hakee solmun korkeuden.*
- int `suurempiLukuVertailu` (int `iLuku1`, int `iLuku2`)
- Vertaa kahta lukua ja palauttaa suuremman.*
- `PUU * poistaSolmu` (char `*pNimi`, `PUU *pJuuriSolmu`)
- Lehtisolmun poistaminen.*
- int `onkoLuku` (char `*pNimi`)
- Testaa, onko käyttäjän antama arvo nimi vai numeroarvo.*
- int `nimenArvo` (char `*pNimi`, `PUU *pJuuriSolmu`)
- Etsii nimen perusteella sen arvon.*
- `PUU * etsiPienin` (`PUU *pUusiSolmu`)
- Etsii pienimmän solmun arvon binääripuusta.*
- `PUU * paivitaPuu` (`PUU *pJuuriSolmu`)
- Tasapainotetaan puu poiston jälkeen.*
- void `kirjoitaBinaaripuu` (char `*pNimi`, `PUU *pAlku`)
- Tasapainoitettun binaaripuun kirjoittaminen tiedostoon.*
- void `kirjoitaTiedostoon` (char `*pKirjoitaTiedostoNimi`, `PUU *pAlku`)
- Kirjoittaa binaaripuun tiedostoon.*
- `RBSOLMU * varaaMuistiaRB` (char `*pNimi`, int `iArvo`)
- Punamustapuu, ehkä oma header olisi kätevämpi, kun omat aliohjelmat oman structin takia.*
- `RBSOLMU * vapautaMuistiRB` (`RBSOLMU *pJuuriSolmu`)
- `RBSOLMU * lisaaRBSolmu` (`RBSOLMU *pJuuriSolmu`, `RBSOLMU *pUusi`)
- `RBSOLMU * luoRBPuu` (`RBSOLMU *pJuuriSolmu`, char `*pNimi`)
- void `kierraVasemmalle` (`RBSOLMU **pJuuriSolmu`, `RBSOLMU *pSolmu`)
- void `kierraOikealle` (`RBSOLMU **pJuuriSolmu`, `RBSOLMU *pSolmu`)
- void `korjaaLisays` (`RBSOLMU **pJuuriSolmu`, `RBSOLMU *pUusi`)
- void `kirjoitaRBTiedostoon` (char `*pNimi`, `RBSOLMU *pJuuriSolmu`)
- void `kirjoitaRB` (char `*pNimi`, `RBSOLMU *pJuuriSolmu`)

4.2.1 Typedef Documentation

4.2.1.1 PUU

```
typedef struct puu PUU
```

4.2.1.2 RBSOLMU

```
typedef struct RBSolmu RBSOLMU
```

4.2.2 Function Documentation

4.2.2.1 etsiPienin()

```
PUU* etsiPienin (
    PUU * pUusiSolmu )
```

Parameters

<i>pUusiSolmu</i>	Osoitin kasiteltavaan solmuun.
-------------------	--------------------------------

Returns

PUU* Palauttaa osoittimen puun pienimpaan solmuun.

Definition at line 447 of file binaaripuu.c.

```
447     {
448         PUU *pNykyinenSolmu = pUusiSolmu;
449
450         while (pNykyinenSolmu->pVasen != NULL) {
451             pNykyinenSolmu = pNykyinenSolmu->pVasen;
452         }
453
454         return (pNykyinenSolmu);
455     }
```

4.2.2.2 kierraOikealle()

```
void kierraOikealle (
    RBSOLMU ** pJuurisolmu,
    RBSOLMU * pSolmu )
```

Definition at line 130 of file punamustapuu.c.

```
130     {
131         RBSOLMU *pVasenLapsi = pSolmu->pVasen;
132         pSolmu->pVasen = pVasenLapsi->pOikea;
133
134         if (pSolmu->pVasen != NULL)
135             pSolmu->pVasen->pVanhempi = pSolmu;
136
137         pVasenLapsi->pVanhempi = pSolmu->pVanhempi;
138
139         if (pSolmu->pVanhempi == NULL)
140             *pJuurisolmu = pVasenLapsi;
141         else if (pSolmu == pSolmu->pVanhempi->pVasen)
142             pSolmu->pVanhempi->pVasen = pVasenLapsi;
143         else
144             pSolmu->pVanhempi->pOikea = pVasenLapsi;
145
146         pVasenLapsi->pOikea = pSolmu;
147         pSolmu->pVanhempi = pVasenLapsi;
148     }
```


4.2.2.3 kierraVasemmalle()

```
void kierraVasemmalle (
    RBSOLMU ** pJuurisolmu,
    RBSOLMU * pSolmu )
```

Definition at line 109 of file punamustapuu.c.

```
109
110     RBSOLMU *pOikeaLapsi = pSolmu->pOikea;
111     pSolmu->pOikea = pOikeaLapsi->pVasen;
112
113     if (pSolmu->pOikea != NULL)
114         pSolmu->pOikea->pVanhempi = pSolmu;
115
116     // Solmun oikea lapsi nousee ylös puussa
117     pOikeaLapsi->pVanhempi = pSolmu->pVanhempi;
118
119     if (pSolmu->pVanhempi == NULL)
120         *pJuurisolmu = pOikeaLapsi;
121     else if (pSolmu == pSolmu->pVanhempi->pVasen)
122         pSolmu->pVanhempi->pVasen = pOikeaLapsi;
123     else
124         pSolmu->pVanhempi->pOikea = pOikeaLapsi;
125
126     pOikeaLapsi->pVasen = pSolmu;
127     pSolmu->pVanhempi = pOikeaLapsi;
128 }
```

4.2.2.4 kirjoitaBinaaripuu()

```
void kirjoitaBinaaripuu (
    char * pNimi,
    PUU * pAlku )
```

Parameters

<i>pNimi</i>	Kirjoitettavan tiedoston nimi.
<i>pAlku</i>	

Definition at line 176 of file binaaripuu.c.

```
176
177     /*if (pAlku == NULL) { //Noora kommentoi pois 19.3.2026 klo. 13.08
178         printf("Puu on tyhjä, luo puurakenne ennen kirjoittamista.\n");
179         return;
180     }*/
181     if (pAlku != NULL) {
182         kirjoitaTiedostoon(pNimi, pAlku);
183         kirjoitaBinaaripuu(pNimi, pAlku->pVasen);
184         kirjoitaBinaaripuu(pNimi, pAlku->pOikea);
185     }
186     return;
187 }
```

4.2.2.5 kirjoitaRB()

```
void kirjoitaRB (
    char * pNimi,
    RBSOLMU * pJuurisolmu )
```

Definition at line 222 of file punamustapuu.c.

```

222                                     {
223     /*if (pJuurisolmu == NULL) {
224         printf("Puu on tyhjä, luo puurakenne ennen kirjoittamista.\n");
225         return;
226     }*/
227     if (pJuurisolmu != NULL) {
228         kirjoitaRBTiedostoon(pNimi, pJuurisolmu);
229         kirjoitaRB(pNimi, pJuurisolmu->pVasen);
230         kirjoitaRB(pNimi, pJuurisolmu->pOikea);
231     }
232     return;
233 }
```

4.2.2.6 kirjoitaRBTiedostoon()

```

void kirjoitaRBTiedostoon (
    char * pNimi,
    RBSOLMU * pJuurisolmu )
```

Definition at line 201 of file punamustapuu.c.

```

201                                     {
202     FILE *Tiedosto = NULL;
203
204     if (pJuurisolmu == NULL) {
205         return;
206     }
207
208     /* Tiedoston avaaminen. */
209     if ((Tiedosto = fopen(pNimi, "a")) == NULL) {
210         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
211         exit(0);
212     }
213     /* Tiedostoon kirjoittaminen. */
214     fprintf(Tiedosto, "%s,%d\n", pJuurisolmu->aNimi, pJuurisolmu->iArvo);
215
216     /* Tiedoston sulkeminen. */
217     fclose(Tiedosto);
218     printf("Tiedosto '%s' kirjoitettu.\n", pNimi);
219     return;
220 }
```

4.2.2.7 kirjoitaTiedostoon()

```

void kirjoitaTiedostoon (
    char * pNimi,
    PUU * pAlku )
```

Parameters

<i>pNimi</i>	Kirjoitettavan tiedoston nimi
<i>pAlku</i>	Solmu, joka kirjoitetaan tiedostoon

Definition at line 195 of file binaaripuu.c.

```

195                                     {
196     FILE *Tiedosto = NULL;
197
198     if (pAlku == NULL) {
199         return;
200     }
201
202     /* Tiedoston avaaminen. */
```

```

203     if ((Tiedosto = fopen(pNimi, "a")) == NULL) {
204         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
205         exit(0);
206     }
207     /* Tiedostoon kirjoittaminen. */
208     fprintf(Tiedosto, "%s,%d\n", pAlku->aNimi, pAlku->iArvo);
209
210     /* Tiedoston sulkeminen. */
211     fclose(Tiedosto);
212     return;
213 }

```

4.2.2.8 korjaaLisays()

```

void korjaaLisays (
    RBSOLMU ** pJuurisolmu,
    RBSOLMU * pUusi )

```

Definition at line 150 of file punamustapuu.c.

```

150
151     // Ilman p-alkua, koska pVanhempi->pVanhempi oli sekava
152     RBSOLMU *vanhempi = NULL;
153     RBSOLMU *seta = NULL;
154     RBSOLMU *isoVanhempi = NULL;
155
156     while ((pUusi != *pJuurisolmu) && (pUusi->pVanhempi->iVariBitti == 0)) {
157         vanhempi = pUusi->pVanhempi;
158         isoVanhempi = vanhempi->pVanhempi;
159
160         if (vanhempi == isoVanhempi->pVasen) {
161             seta = isoVanhempi->pOikea;
162             if (seta != NULL && seta->iVariBitti == 0) {
163                 vanhempi->iVariBitti = 1;
164                 seta->iVariBitti = 1;
165                 isoVanhempi->iVariBitti = 0;
166                 pUusi = isoVanhempi;
167             } else {
168                 if (pUusi == vanhempi->pOikea) {
169                     pUusi = vanhempi;
170                     kierraVasemmalle(pJuurisolmu, pUusi);
171                     vanhempi = pUusi->pVanhempi;
172                 }
173                 vanhempi->iVariBitti = 1;
174                 isoVanhempi->iVariBitti = 0;
175                 kierraOikealle(pJuurisolmu, isoVanhempi);
176             }
177
178         } else {
179             seta = isoVanhempi->pVasen;
180             if (seta != NULL && seta->iVariBitti == 0) {
181                 vanhempi->iVariBitti = 1;
182                 seta->iVariBitti = 1;
183                 isoVanhempi->iVariBitti = 0;
184                 pUusi = isoVanhempi;
185             } else {
186                 if (pUusi == vanhempi->pVasen) {
187                     pUusi = vanhempi;
188                     kierraOikealle(pJuurisolmu, pUusi);
189                     vanhempi = pUusi->pVanhempi;
190                 }
191                 vanhempi->iVariBitti = 1;
192                 isoVanhempi->iVariBitti = 0;
193                 kierraVasemmalle(pJuurisolmu, isoVanhempi);
194             }
195         }
196     }
197     (*pJuurisolmu)->iVariBitti = 1;
198 }

```

4.2.2.9 lisaaRBSolmu()

```
RBSOLMU* lisaaRBSolmu (
    RBSOLMU * pJuurisolmu,
    RBSOLMU * pUusi )
```

Katsotaan, ovatko arvot samat. Laitetaan aakkosten perusteella vasemmalle tai oikealle.

Definition at line 38 of file punamustapuu.c.

```
38                                     {
39     int iVertailu = 0;
40
41     // Jos puu on tyhjä, palautetaan uusi RBSolmu.
42     if (pJuurisolmu == NULL) {
43         return pUusi;
44     }
45
46     iVertailu = strcmp(pUusi->aNimi, pJuurisolmu->aNimi);
47
48     // Solmujen lisääminen rekursiivisesti
49     if (pUusi->iArvo < pJuurisolmu->iArvo) {
50         pJuurisolmu->pVasen = lisaaRBSolmu(pJuurisolmu->pVasen, pUusi);
51         pJuurisolmu->pVasen->pVanhempi = pJuurisolmu;
52     } else if (pUusi->iArvo > pJuurisolmu->iArvo) {
53         pJuurisolmu->pOikea = lisaaRBSolmu(pJuurisolmu->pOikea, pUusi);
54         pJuurisolmu->pOikea->pVanhempi = pJuurisolmu;
55     } else if (pUusi->iArvo == pJuurisolmu->iArvo) {
56         if (iVertailu < 0) {
57             pJuurisolmu->pVasen = lisaaRBSolmu(pJuurisolmu->pVasen, pUusi);
58             pJuurisolmu->pVasen->pVanhempi = pJuurisolmu;
59         } else if (iVertailu > 0) {
60             pJuurisolmu->pOikea = lisaaRBSolmu(pJuurisolmu->pOikea, pUusi);
61             pJuurisolmu->pOikea->pVanhempi = pJuurisolmu;
62         }
63     }
64     return (pJuurisolmu);
65 }
66
67 }
```

4.2.2.10 lisaaSolmu()

```
PUU* lisaaSolmu (
    PUU * pAlku,
    char * pSolmu,
    int iArvo )
```

Tasapainottaa puun.

Parameters

<i>pAlku</i>	
<i>pSolmu</i>	Lisattava solmu.
<i>iArvo</i>	Solmun arvo.

Returns

PUU*

Definition at line 58 of file binaaripuu.c.

```
58                                     {
59     int iVertailu = 0;
60
61     // Varataan muistia, jos muisti tyhjä.
```

```

62     if (pAlku == NULL) {
63         return varaaMuistiaPuulle(pSolmu, iArvo);
64     }
65
66     iVertailu = strcmp(pSolmu, pAlku->aNimi);
67
68     // Solmujen lisääminen.
69     if (iArvo < pAlku->iArvo) {
70         pAlku->pVasen = lisaaSolmu(pAlku->pVasen, pSolmu, iArvo);
71     } else if (iArvo > pAlku->iArvo) {
72         pAlku->pOikea = lisaaSolmu(pAlku->pOikea, pSolmu, iArvo);
73     }
74
75     // Testataan, ovatko arvot samat.
76     if (iVertailu < 0) {
77         pAlku->pVasen = lisaaSolmu(pAlku->pVasen, pSolmu, iArvo);
78     } else if (iVertailu > 0) {
79         pAlku->pOikea = lisaaSolmu(pAlku->pOikea, pSolmu, iArvo);
80     }
81 }
82
83 // Selvitetaan paikka, johon solmu lisataan. Tasapainotetaan puu.
84 pAlku->iPituus = 1 + suurempiLukuVertailu(puunPituus(pAlku->pVasen), puunPituus(pAlku->pOikea));
85 int tasapaino = tasapainoitaPuu(pAlku);
86
87 // vasen vasen tasapainotus
88 if ((tasapaino > 1) && (iArvo < pAlku->pVasen->iArvo)) {
89     return (oikeaPuoli(pAlku));
90 }
91
92 // vasen oikea tasapainotus
93 if ((tasapaino > 1) && (iArvo > pAlku->pVasen->iArvo)) {
94     pAlku->pVasen = vasenPuoli(pAlku->pVasen);
95     return (oikeaPuoli(pAlku));
96 }
97
98 // oikea vasen tasapainotus
99 if ((tasapaino < -1) && (iArvo < pAlku->pOikea->iArvo)) {
100     pAlku->pOikea = oikeaPuoli(pAlku->pOikea);
101     return (vasenPuoli(pAlku));
102 }
103
104 // oikea oikea tasapainotus
105 if ((tasapaino < -1) && (iArvo > pAlku->pOikea->iArvo)) {
106     return (vasenPuoli(pAlku));
107 }
108
109 return (pAlku);
110 }

```

4.2.2.11 luoPuu()

```

PUU* luoPuu (
    char * pNimi,
    PUU * pJuuriSolmu )

```

Pilkotaan luetut rivit.

Parameters

<i>pNimi</i>	Luettavan tiedoston nimi.
<i>pJuuriSolmu</i>	Puun juurisolmu.

Returns

PUU* Palauttaa juuriSolmun.

Definition at line 119 of file binaaripuu.c.

```

119                                     {
120     FILE *Tiedosto = NULL;
121     char aRivi[LEN] = "";
122     int iOtsikko = 0;
123     char *p1 = NULL, *p2 = NULL;
124     int iArvo = 0;
125
126     // Tiedoston avaus
127     if ((Tiedosto = fopen(pNimi, "r")) == NULL) {
128         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
129         exit(0);
130     }
131
132     // Tiedoston lukeminen
133     while ((fgets(aRivi, LEN, Tiedosto)) != NULL) {
134         aRivi[strlen(aRivi) - 1] = '\\0';
135
136         // Ohitetaan otsikko
137         if (iOtsikko == 0) {
138             iOtsikko++;
139             continue;
140         }
141
142         // Pilkotaan rivit
143         if ((p1 = strtok(aRivi, ";")) == NULL) {
144             perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
145             exit(0);
146         }
147
148         if ((p2 = strtok(NULL, ";")) == NULL) {
149             perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
150             exit(0);
151         }
152
153         iArvo = atoi(p2);
154
155         // Maaritetaan juuri solmu jos sitä ei ole määriteltty.
156         if (pJuuriSolmu == NULL) {
157             pJuuriSolmu = lisaaSolmu(pJuuriSolmu, p1, iArvo);
158             iOtsikko++;
159             continue;
160         }
161
162         pJuuriSolmu = lisaaSolmu(pJuuriSolmu, p1, iArvo);
163     }
164
165     // Suljetaan tiedosto
166     fclose(Tiedosto);
167     return (pJuuriSolmu);
168 }

```

4.2.2.12 `luoRBPuu()`

```

RBSOLMU* luoRBPuu (
    RBSOLMU * pJuurisolmu,
    char * pNimi )

```

Definition at line 69 of file punamustapuu.c.

```

69                                     {
70     FILE *Tiedosto = NULL;
71     char aRivi[LEN] = "";
72     int iOtsikko = 0;
73     char *p1 = NULL, *p2 = NULL;
74     int iArvo = 0;
75
76     if ((Tiedosto = fopen(pNimi, "r")) == NULL) {
77         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
78         exit(0);
79     }
80
81     while ((fgets(aRivi, LEN, Tiedosto)) != NULL) {
82         aRivi[strlen(aRivi) - 1] = '\\0';
83         if (iOtsikko == 0) {
84             iOtsikko++;
85             continue;
86         }
87
88         if ((p1 = strtok(aRivi, ";")) == NULL) {

```

```

89         perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
90         exit(0);
91     }
92
93     if ((p2 = strtok(NULL, ";")) == NULL) {
94         perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
95         exit(0);
96     }
97
98     iArvo = atoi(p2);
99
100     RBSOLMU *pUusi = varaaMuistiaRB(pl, iArvo);
101     pJuuriSolmu = lisaaRBSolmu(pJuuriSolmu, pUusi);
102     korjaaLisays(&pJuuriSolmu, pUusi);
103 }
104
105 fclose(Tiedosto);
106 return (pJuuriSolmu);
107 }

```

4.2.2.13 nimenArvo()

```

int nimenArvo (
    char * pNimi,
    PUU * pJuuriSolmu )

```

Parameters

<i>pNimi</i>	Poistettava nimi merkkijonona.
<i>pJuuriSolmu</i>	Osoitin kasiteltavaan solmuun.

Returns

int Palauttaa 0 tai 1, riippuen siitä, löytyykö nimen nimen pohjalta arvoa.

Definition at line 422 of file binaaripuu.c.

```

422                                     {
423     if (pJuuriSolmu == NULL) {
424         return (0);
425     }
426
427     if (strcmp(pJuuriSolmu->aNimi, pNimi) == 0) {
428         int iArvo = pJuuriSolmu->iArvo;
429         return (iArvo);
430     }
431
432     int iArvo = nimenArvo(pNimi, pJuuriSolmu->pVasen);
433     if (iArvo != 0) {
434         return (iArvo);
435     } else {
436         int iArvo = nimenArvo(pNimi, pJuuriSolmu->pOikea);
437         return (iArvo);
438     }
439 }

```

4.2.2.14 oikeaPuoli()

```

PUU* oikeaPuoli (
    PUU * pAlku )

```

Puu järjestetään, jotta se on tasapainossa.

Parameters

<i>pAlku</i>	
--------------	--

Returns

PUU* Palauttaa solmun.

Definition at line 237 of file binaaripuu.c.

```

237     {
238         // Tarkistetaan, onko pAlku tai pAlku oikea puoli NULL
239         if ((pAlku->pVasen == NULL) || (pAlku == NULL)) {
240             return (pAlku);
241         }
242
243         // Muuttujien ja vakioiden alustaminen
244         PUU *pMuuttujal = pAlku->pVasen;
245         PUU *pMuuttuja2 = pMuuttujal->pOikea;
246
247         // Sijoitetetaan arvoja
248         pMuuttujal->pOikea = pAlku;
249         pAlku->pVasen = pMuuttuja2;
250
251         // Verrataan ja sijoitetaan suuremmat korkeudet
252         pAlku->iPituus = suurempiLukuVertailu(puunPituus(pAlku->pVasen), puunPituus(pAlku->pOikea)) + 1;
253         pMuuttujal->iPituus =
254             suurempiLukuVertailu(puunPituus(pMuuttujal->pVasen), puunPituus(pMuuttujal->pOikea)) + 1;
255
256         return pMuuttujal;
257     }

```

4.2.2.15 onkoLuku()

```

int onkoLuku (
    char * pNimi )

```

Ilmoittaa, onko käyttäjän antama numero vai ei.

Parameters

<i>pNimi</i>	Poistettava nimi tai numeroarvo merkkijonona.
--------------	---

Returns

int Palauttaa 0 tai 1, riippuen siitä onko käyttäjän syöte numero.

Definition at line 401 of file binaaripuu.c.

```

401     {
402         int tosi = 0;
403         char *ptr = pNimi;
404
405         while (*ptr) {
406             if (!isdigit(*ptr)) {
407                 return tosi;
408             }
409             ptr++;
410         }
411         tosi = 1;
412         return (tos);
413     }

```


4.2.2.16 paivitaPuu()

```
PUU* paivitaPuu (
    PUU * pJuuriSolmu )
```

Parameters

<i>pJuuriSolmu</i>	Osoitin kasiteltavaan solmuun.
--------------------	--------------------------------

Returns

PUU* palauttaa tasapainotetun solmun.

Definition at line 463 of file binaaripuu.c.

```
463 {
464     // Puun korkeuden päivittäminen.
465     if (pJuuriSolmu == NULL) {
466         return (pJuuriSolmu);
467     }
468
469     int iLuku1 = puunPituus(pJuuriSolmu->pVasen);
470     int iLuku2 = puunPituus(pJuuriSolmu->pOikea);
471     pJuuriSolmu->iPituus = suurempiLukuVertailu(iLuku1, iLuku2) + 1;
472
473     // Puun tasapanottaminen.
474     int iTasapaino = tasapainoitaPuu(pJuuriSolmu);
475     // Vasen-vasen tapaus.
476     if (iTasapaino > 1 && tasapainoitaPuu(pJuuriSolmu->pVasen) >= 0) {
477         return oikeaPuoli(pJuuriSolmu);
478     }
479
480     // Oikea-oikea tapaus.
481     if (iTasapaino < -1 && tasapainoitaPuu(pJuuriSolmu->pOikea) <= 0) {
482         return vasenPuoli(pJuuriSolmu);
483     }
484
485     // Vasen-oikea tapaus.
486     if (iTasapaino > 1 && tasapainoitaPuu(pJuuriSolmu->pVasen) < 0) {
487         pJuuriSolmu->pVasen = vasenPuoli(pJuuriSolmu->pVasen);
488         return oikeaPuoli(pJuuriSolmu);
489     }
490
491     // Oikea-vasen tapaus.
492     if (iTasapaino < -1 && tasapainoitaPuu(pJuuriSolmu->pOikea) > 0) {
493         pJuuriSolmu->pOikea = oikeaPuoli(pJuuriSolmu->pOikea);
494         return vasenPuoli(pJuuriSolmu);
495     }
496
497     return (pJuuriSolmu);
498 }
```

4.2.2.17 poistaSolmu()

```
PUU* poistaSolmu (
    char * pNimi,
    PUU * pJuuriSolmu )
```

Poistaa lehtisolmun joko numeroarvon tai nimen perusteella.

Parameters

<i>pNimi</i>	Poistettava nimi tai numeroarvo merkkijonona.
<i>pJuuriSolmu</i>	Osoitin kasiteltavaan puun solmuun.
<i>iArvo</i>	Haettava numeroarvo.

Returns

PUU* Uusi osoitin puun solmuun, NULL jos puu tyhjeni.

Definition at line 334 of file binaaripuu.c.

```

334                                     {
335     int iArvo = 0;
336     PUU *pUusiSolmu = NULL;
337
338     if (pJuuriSolmu == NULL) {
339         printf("Solmua ei löytynyt.\n");
340         return (pJuuriSolmu);
341     }
342
343     // Testataan, onko syöte nimi vai lukuarvo, ja määritetään lukuarvo puun läpikäymiseksi.
344     if (onkoLuku(pNimi)) {
345         iArvo = atoi(pNimi);
346     } else {
347         iArvo = nimenArvo(pNimi, pJuuriSolmu);
348
349         if (iArvo == 0) {
350             printf("Solmua ei löytynyt tällä nimellä.\n");
351             return (pJuuriSolmu);
352         }
353     }
354
355     // Lehtisolmun poistaminen.
356     if (iArvo < pJuuriSolmu->iArvo) {
357         pJuuriSolmu->pVasen = poistaSolmu(pNimi, pJuuriSolmu->pVasen);
358     } else if (iArvo > pJuuriSolmu->iArvo) {
359         pJuuriSolmu->pOikea = poistaSolmu(pNimi, pJuuriSolmu->pOikea);
360     } else {
361
362         if (pJuuriSolmu->pVasen == NULL && pJuuriSolmu->pOikea == NULL) {
363             free(pJuuriSolmu);
364             pJuuriSolmu = NULL;
365             printf("Solmu poistettu.\n");
366             return (pJuuriSolmu);
367
368         } else if (pJuuriSolmu->pVasen == NULL || pJuuriSolmu->pOikea == NULL) {
369             if (pJuuriSolmu->pVasen != NULL) {
370                 pUusiSolmu = pJuuriSolmu->pVasen;
371             } else {
372                 pUusiSolmu = pJuuriSolmu->pOikea;
373             }
374             free(pJuuriSolmu);
375             pJuuriSolmu = NULL;
376             printf("Solmu poistettu.\n");
377             return (pUusiSolmu);
378
379         } else {
380             pUusiSolmu = etsiPienin(pJuuriSolmu->pOikea);
381             pJuuriSolmu->iArvo = pUusiSolmu->iArvo;
382             strcpy(pJuuriSolmu->aNimi, pUusiSolmu->aNimi);
383             pJuuriSolmu->pOikea = poistaSolmu(pUusiSolmu->aNimi, pJuuriSolmu->pOikea);
384             return (pJuuriSolmu);
385         }
386     }
387
388     pJuuriSolmu = paivitaPuu(pJuuriSolmu);
389     return (pJuuriSolmu);
390 }

```

4.2.2.18 puunPituus()

```

int puunPituus (
    PUU * pAlku )

```

Parameters

<i>pAlku</i>	Kohta, josta korkeus haetaan
--------------	------------------------------

Returns

int Palauttaa solmun korkeuden

Definition at line 293 of file binaaripuu.c.

```

293     {
294         // Tarkistetaan, onko Solmu Null
295         if (pAlku == NULL) {
296             return (0);
297         }
298
299         // Palauttaa solmun korkeuden
300         return (pAlku->iPituus);
301     }
```

4.2.2.19 suurempiLukuVertailu()

```

int suurempiLukuVertailu (
    int iLuku1,
    int iLuku2 )
```

Parameters

<i>iLuku1</i>	Ensimmäinen verrattava kokonaisluku.
<i>iLuku2</i>	Toinen verrattava kokonaisluku.

Returns

int Palauttaa suuremman luvun.

Definition at line 310 of file binaaripuu.c.

```

310     {
311         int iPalautus = 0;
312
313         // Verrataan kumpi luvuista on suurempi.
314         if (iLuku1 > iLuku2) {
315             iPalautus = iLuku1;
316         } else {
317             iPalautus = iLuku2;
318         }
319
320         // Palauttaa suuremman luvun.
321         return (iPalautus);
322     }
```

4.2.2.20 tasapainoitaPuu()

```

int tasapainoitaPuu (
    PUU * pAlku )
```

Parameters

<i>pAlku</i>	Solmu, josta tasapainokerroin halutaan
--------------	--

Returns

int

Definition at line 221 of file binaaripuu.c.

```

221      {
222      // Tarkistetaan, onko Null
223      if (pAlku == NULL) {
224          return (0);
225      }
226
227      // Palautetaan tasapainokerroin
228      return (puunPituus(pAlku->pVasen) - puunPituus(pAlku->pOikea));
229  }
```

4.2.2.21 vapautaMuistiPuu()

```

PUU* vapautaMuistiPuu (
    PUU * pJuuriSolmu )
```

Parameters

<i>pJuuriSolmu</i>	Puun juurisolmu
--------------------	-----------------

Returns

PUU*

Definition at line 39 of file binaaripuu.c.

```

39      {
40      // Muistin vapauttaminen
41      if (pJuuriSolmu != NULL) {
42          vapautaMuistiPuu(pJuuriSolmu->pVasen);
43          vapautaMuistiPuu(pJuuriSolmu->pOikea);
44          free(pJuuriSolmu);
45      }
46      pJuuriSolmu = NULL;
47      return (pJuuriSolmu);
48  }
```

4.2.2.22 vapautaMuistiRB()

```

RBSOLMU* vapautaMuistiRB (
    RBSOLMU * pJuuriSolmu )
```

Definition at line 28 of file punamustapuu.c.

```

28      {
29      if (pJuuriSolmu != NULL) {
30          vapautaMuistiRB(pJuuriSolmu->pVasen);
31          vapautaMuistiRB(pJuuriSolmu->pOikea);
32          free(pJuuriSolmu);
33      }
34      pJuuriSolmu = NULL;
35      return (pJuuriSolmu);
36  }
```

4.2.2.23 varaaMuistiaPuulle()

```
PUU* varaaMuistiaPuulle (
    char * pSolmu,
    int iArvo )
```

Parameters

<i>pSolmu</i>	Solmu, jolle muistia varataan
<i>iArvo</i>	Arvo, jolle muistia varataan

Returns

PUU*

Definition at line 16 of file binaaripuu.c.

```

16                                     {
17     PUU *pUusi = NULL;
18
19     // Muistin varaaminen
20     if ((pUusi = (PUU *)malloc(sizeof(PUU))) == NULL) {
21         perror("Muistin varaus epäonnistui, lopetetaan");
22         exit(0);
23     }
24
25     strcpy(pUusi->aNimi, pSolmu);
26     pUusi->iArvo = iArvo;
27     pUusi->iPituus = 1;
28     pUusi->pVasen = NULL;
29     pUusi->pOikea = NULL;
30     return (pUusi);
31 }
```

4.2.2.24 varaaMuistiaRB()

```

RBSOLMU* varaaMuistiaRB (
    char * pNimi,
    int iArvo )
```

Punamustapuu, ehkä oma header olisi kätevämpi, kun omat aliohjelmat oman structin takia.

Definition at line 11 of file punamustapuu.c.

```

11                                     {
12     RBSOLMU *pUusi = NULL;
13
14     if ((pUusi = (RBSOLMU *)malloc(sizeof(RBSOLMU))) == NULL) {
15         perror("Muistin varaus epäonnistui, lopetetaan");
16         exit(0);
17     }
18
19     pUusi->iVariBitti = 0; // uudet aina punaisia
20     strcpy(pUusi->aNimi, pNimi);
21     pUusi->iArvo = iArvo;
22     pUusi->pVasen = NULL;
23     pUusi->pOikea = NULL;
24     pUusi->pVanhempi = NULL;
25     return (pUusi);
26 }
```

4.2.2.25 vasenPuoli()

```

PUU* vasenPuoli (
    PUU * pAlku )
```

Puu järjestetään, jotta se on tasapainossa.

Parameters

<i>pAlku</i>	
--------------	--

Returns

PUU* Palauttaa solmun.

Definition at line 265 of file binaaripuu.c.

```

265         {
266     // Tarkistetaan, onko pAlku tai pAlku oikea puoli NULL
267     if ((pAlku->pOikea == NULL) || (pAlku == NULL)) {
268         return (pAlku);
269     }
270
271     // Muuttujien ja vakioiden alustaminen
272     PUU *pMuuttujal = pAlku->pOikea;
273     PUU *pMuuttuja2 = pMuuttujal->pVasen;
274
275     // Sijoitetaan arvoja
276     pMuuttujal->pVasen = pAlku;
277     pAlku->pOikea = pMuuttuja2;
278
279     // Verrataan ja sijoitetaan suuremmat korkeudet
280     pAlku->iPituus = suurempiLukuVertailu(puunPituus(pAlku->pVasen), puunPituus(pAlku->pOikea)) + 1;
281     pMuuttujal->iPituus =
282         suurempiLukuVertailu(puunPituus(pMuuttujal->pVasen), puunPituus(pMuuttujal->pOikea)) + 1;
283
284     return pMuuttujal;
285 }
```

4.3 haut.c File Reference

```

#include "haut.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

Functions

- int [syvyyshaku](#) (char *pNimi, [PUU](#) *pAlku, int iArvo)
Tekee syvyysshaun numeroarvon pohjalta.
- void [tarkistaLoytyykoSyvyysshaulla](#) (char *aNimiKirjoitettava, [PUU](#) *pJuuriSolmu, int iArvo)
Tarkistaa, loytyyko etsitty arvo syvyysshaussa.
- void [leveyshaku](#) (char *pNimi, [PUU](#) *pJuuriSolmu, char *pHaku)
Tekee leveyshaun nimen pohjalta.
- [JONO](#) * [varaaMuistiaJonolle](#) ([PUU](#) *pSolmu)
Varaa muistia jonolle.
- [JONO](#) * [vapautaMuistiJono](#) ([JONO](#) *pAlku)
Vapauttaa jonon varaaman muistin.
- int [binaarihaku](#) (char *pNimi, [PUU](#) *pJuuriSolmu, int iArvo)
Binaarihaku perustuen numeroarvoon.

4.3.1 Function Documentation

4.3.1.1 binaarihaku()

```
int binaarihaku (
    char * pNimi,
    PUU * pJuuriSolmu,
    int iArvo )
```

Binaarihaku, joka tehdään käyttäjän antaman numeroarvon perusteella.

Parameters

<i>pNimi</i>	Kirjoitettavan tiedoston nimi.
<i>pJuuriSolmu</i>	Osoitin kasiteltavaan puun solmuun.
<i>iArvo</i>	Haettava numeroarvo.

Returns

int Palauttaa joko 0 tai 1, riippuen siitä, löytyykö numeroarvo binaaripuusta.

Definition at line 175 of file haut.c.

```
175                                     {
176
177     if (pJuuriSolmu == NULL) {
178         return (0);
179     }
180
181     kirjoitaTiedostoon(pNimi, pJuuriSolmu);
182
183     if (iArvo == pJuuriSolmu->iArvo) {
184         printf("Hakemasi arvo '%d' löytyi!\n", iArvo);
185         return (1);
186     } else if (iArvo < pJuuriSolmu->iArvo) {
187         return binaarihaku(pNimi, pJuuriSolmu->pVasen, iArvo);
188     } else {
189         return binaarihaku(pNimi, pJuuriSolmu->pOikea, iArvo);
190     }
191 }
```

4.3.1.2 leveyshaku()

```
void leveyshaku (
    char * pNimi,
    PUU * pJuuriSolmu,
    char * pHaku )
```

Tekee leveyshaun käyttäjän antaman nimen pohjalta. Kirjoittaa leveyshaun polun käyttäjän määrittelemään tiedostoon.

Parameters

<i>pNimi</i>	Käyttäjän antama kirjoitettavan tiedoston nimi.
<i>pAlku</i>	Osoitin puu tietueen alkuun, eli ensimmäiseen solmuun.
<i>pHaku</i>	Osoitin haettavan nimen merkkijonoon.

Returns

void

Definition at line 73 of file haut.c.

```

73                                     {
74     // Leveyshaku nimen mukaan, kirjoitetaan käydyt solmut tiedostoon.
75     JONO *pAlku = varaaMuistiaJonolle(pJuuriSolmu);
76     JONO *pLoppu = pAlku;
77     JONO *pEdellinen = NULL;
78     int iVertailu = 0;
79     int iLoytyi = 0;
80
81     if (pJuuriSolmu == NULL) {
82         printf("Puu on tyhjä, luo puurakenne ennen leveyshakua.\n");
83         return;
84     }
85
86     while (pAlku != NULL) {
87         PUU *ptr = pAlku->pSolmu;
88         kirjoitaTiedostoon(pNimi, ptr);
89
90         iVertailu = strcmp(ptr->aNimi, pHaku);
91
92         if (iVertailu == 0) {
93             printf("Nimi '%s' löytyi puusta.\n", ptr->aNimi);
94             iLoytyi = 1;
95             break;
96         }
97
98         if (ptr->pVasen != NULL) {
99             pLoppu->pSeuraava = varaaMuistiaJonolle(ptr->pVasen);
100             pLoppu = pLoppu->pSeuraava;
101         }
102
103         if (ptr->pOikea != NULL) {
104             pLoppu->pSeuraava = varaaMuistiaJonolle(ptr->pOikea);
105             pLoppu = pLoppu->pSeuraava;
106         }
107
108         pEdellinen = pAlku;
109         pAlku = pAlku->pSeuraava;
110         free(pEdellinen);
111     }
112
113     if (iLoytyi == 0) {
114         printf("Hakemaasi nimeä ei löytynyt puusta.\n");
115     }
116     pAlku = vapautaMuistiJono(pAlku);
117     return;
118 }

```

4.3.1.3 syvyyshaku()

```

int syvyyshaku (
    char * pNimi,
    PUU * pAlku,
    int iArvo )

```

Tekee syvyyshaun käyttäjän antaman numeroarvon pohjalta. Kirjoittaa syvyyshaun polun käyttäjän määrittelemään tiedostoon.

Parameters

<i>pNimi</i>	Käyttäjän antama kirjoitettavan tiedoston nimi.
<i>pAlku</i>	Osoitin puu tietueen alkuun, eli ensimmäiseen solmuun.
<i>iArvo</i>	Arvo, jota etsitään syvyyshaussa.

Returns

int Palauttaa arvon 0 tai 1, riippuen siitä, löytyykö etsitty arvo.

Definition at line 17 of file haut.c.

```

17                                     {
18     // Syvyyshaku arvon mukaan, kirjoitetaan käydyt solmut tiedostoon.
19     int iLoytyi = 0;
20
21     if (pAlku == NULL) {
22         return 0;
23     }
24
25     if (pAlku != NULL) {
26         kirjoitaTiedostoon(pNimi, pAlku);
27
28         if (pAlku->iArvo == iArvo) {
29             iLoytyi = 1;
30             printf("Arvo %d löytyi.\n", iArvo);
31         } else {
32             if (iLoytyi == 0) {
33                 iLoytyi = syvyyshaku(pNimi, pAlku->pVasen, iArvo);
34             }
35             if (iLoytyi == 0) {
36                 iLoytyi = syvyyshaku(pNimi, pAlku->pOikea, iArvo);
37             }
38         }
39     }
40     return (iLoytyi);
41 }

```

4.3.1.4 tarkistaLoytyykoSyvyyshauulla()

```

void tarkistaLoytyykoSyvyyshauulla (
    char * aNimiKirjoitettava,
    PUU * pJuuriSolmu,
    int iArvo )

```

Tarkistaa, löytyyko syvyysaussa etsitty arvo. Jos arvoa ei löydy, tulostaa tiedon siitä.

Parameters

<i>aNimiKirjoitettava</i>	Kayttajan antama kirjoitettavan tiedoston nimi.
<i>pJuuriSolmu</i>	Osoitin puu tietueen alkuun, eli ensimmäiseen solmuun.
<i>iArvo</i>	Syvyysaussa haettava numeroarvo.

Returns

void

Definition at line 54 of file haut.c.

```

54                                     {
55     int iLoytyi = 0;
56     iLoytyi = syvyyshaku(aNimiKirjoitettava, pJuuriSolmu, iArvo);
57     if (iLoytyi == 0) {
58         printf("Arvoa %d ei löytynyt puusta.\n", iArvo);
59     }
60 }

```

4.3.1.5 vapautaMuistiJono()

```
JONO* vapautaMuistiJono (
    JONO * pAlku )
```

Vapauttaa jonon alkioita varten varatun muistin.

Parameters

<i>pAlku</i>	Osoitin jonon ensimmäiseen alkioon.
--------------	-------------------------------------

Returns

pAlku Palauttaa NULL, koska jono on tyhjä.

Definition at line 151 of file haut.c.

```
151                                     {
152     // Jonon muistin vapauttaminen.
153     JONO *ptr = pAlku;
154     JONO *pSeuraava = NULL;
155
156     while (ptr != NULL) {
157         pSeuraava = ptr->pSeuraava;
158         free(ptr);
159         ptr = pSeuraava;
160     }
161     pAlku = NULL;
162     return (pAlku);
163 }
```

4.3.1.6 varaaMuistiaJonolle()

```
JONO* varaaMuistiaJonolle (
    PUU * pSolmu )
```

Varaa muistia jonon uudelle alkiolle leveyshakua varten.

Parameters

<i>pSolmu</i>	Osoitin kasiteltavaan puun solmuun.
---------------	-------------------------------------

Returns

pUusi Osoitin uuteen jonon alkioon.

Definition at line 128 of file haut.c.

```
128                                     {
129     JONO *pUusi = NULL;
130
131     // Muistin varaaminen jonolle.
132     if ((pUusi = (JONO *)malloc(sizeof(JONO))) == NULL) {
133         perror("Muistin varaus epäonnistui, lopetetaan");
134         exit(0);
135     }
136
137     pUusi->pSolmu = pSolmu;
138     pUusi->pSeuraava = NULL;
139
140     return (pUusi);
141 }
```

4.4 haut.h File Reference

```
#include "binaaripuu.h"
```

Classes

- struct [jono](#)

Typedefs

- typedef struct [jono](#) [JONO](#)

Functions

- int [syvyyshaku](#) (char *pKirjoitaTiedostoNimi, [PUU](#) *pJuuriSolmu, int iArvo)
Tekee syvyysshaun numeroarvon pohjalta.
- void [tarkistaLoytvykoSyvyysshaulla](#) (char *aNimiKirjoitettava, [PUU](#) *pJuuriSolmu, int iArvo)
Tarkistaa, loytyyko etsitty arvo syvyysshaussa.
- void [leveyshaku](#) (char *pKirjoitaTiedostoNimi, [PUU](#) *pAlku, char *pHaku)
Tekee leveyshaun nimen pohjalta.
- [JONO](#) * [varaaMuistiaJonolle](#) ([PUU](#) *pSolmu)
Varaa muistia jonolle.
- [JONO](#) * [vapautaMuistiJono](#) ([JONO](#) *pAlku)
Vapauttaa jonon varaaman muistin.
- int [binaarihaku](#) (char *pNimi, [PUU](#) *pJuuriSolmu, int iArvo)
Binaarihaku perustuen numeroarvoon.

4.4.1 Typedef Documentation

4.4.1.1 JONO

```
typedef struct jono JONO
```

4.4.2 Function Documentation

4.4.2.1 binaarihaku()

```
int binaarihaku (  
    char * pNimi,  
    PUU * pJuuriSolmu,  
    int iArvo )
```

Binaarihaku, joka tehdään käyttäen antaman numeroarvon perusteella.

Parameters

<i>pNimi</i>	Kirjoitettavan tiedoston nimi.
<i>pJuuriSolmu</i>	Osoitin kasiteltavaan puun solmuun.
<i>iArvo</i>	Haettava numeroarvo.

Returns

int Palauttaa joko 0 tai 1, riippuen siitä, löytyykö numeroarvo binaaripuusta.

Definition at line 175 of file haut.c.

```

175                                     {
176
177     if (pJuuriSolmu == NULL) {
178         return (0);
179     }
180
181     kirjoitaTiedostoon(pNimi, pJuuriSolmu);
182
183     if (iArvo == pJuuriSolmu->iArvo) {
184         printf("Hakemasi arvo '%d' löytyi!\n", iArvo);
185         return (1);
186     } else if (iArvo < pJuuriSolmu->iArvo) {
187         return binaarihaku(pNimi, pJuuriSolmu->pVasen, iArvo);
188     } else {
189         return binaarihaku(pNimi, pJuuriSolmu->pOikea, iArvo);
190     }
191 }
```

4.4.2.2 leveyshaku()

```

void leveyshaku (
    char * pNimi,
    PUU * pJuuriSolmu,
    char * pHaku )
```

Tekee leveyshaun käyttäjän antaman nimen pohjalta. Kirjoittaa leveyshaun polun käyttäjän määrittelemään tiedostoon.

Parameters

<i>pNimi</i>	Käyttäjän antama kirjoitettavan tiedoston nimi.
<i>pAlku</i>	Osoitin puu tietueen alkuun, eli ensimmäiseen solmuun.
<i>pHaku</i>	Osoitin haettavan nimen merkkijonoon.

Returns

void

Definition at line 73 of file haut.c.

```

73                                     {
74     // Leveyshaku nimen mukaan, kirjoitetaan käydyt solmut tiedostoon.
75     JONO *pAlku = varaaMuistiaJonolle(pJuuriSolmu);
76     JONO *pLoppu = pAlku;
77     JONO *pEdellinen = NULL;
78     int iVertailu = 0;
79     int iLöytyi = 0;
80 }
```

```

81     if (pJuuriSolmu == NULL) {
82         printf("Puu on tyhjä, luo puurakenne ennen leveyshakua.\n");
83         return;
84     }
85
86     while (pAlku != NULL) {
87         PUU *ptr = pAlku->pSolmu;
88         kirjoitaTiedostoon(pNimi, ptr);
89
90         iVertailu = strcmp(ptr->aNimi, pHaku);
91
92         if (iVertailu == 0) {
93             printf("Nimi '%s' löytyi puusta.\n", ptr->aNimi);
94             iLoytyi = 1;
95             break;
96         }
97
98         if (ptr->pVasen != NULL) {
99             pLoppu->pSeuraava = varaaMuistiaJonolle(ptr->pVasen);
100             pLoppu = pLoppu->pSeuraava;
101         }
102
103         if (ptr->pOikea != NULL) {
104             pLoppu->pSeuraava = varaaMuistiaJonolle(ptr->pOikea);
105             pLoppu = pLoppu->pSeuraava;
106         }
107
108         pEdellinen = pAlku;
109         pAlku = pAlku->pSeuraava;
110         free(pEdellinen);
111     }
112
113     if (iLoytyi == 0) {
114         printf("Hakemaasi nimeä ei löytynyt puusta.\n");
115     }
116     pAlku = vapautaMuistiJono(pAlku);
117     return;
118 }

```

4.4.2.3 syvyyshaku()

```

int syvyyshaku (
    char * pNimi,
    PUU * pAlku,
    int iArvo )

```

Tekee syvyysshaun käyttäjän antaman numeroarvon pohjalta. Kirjoittaa syvyysshaun polun käyttäjän määrittelemään tiedostoon.

Parameters

<i>pNimi</i>	Käyttäjän antama kirjoitettavan tiedoston nimi.
<i>pAlku</i>	Osoitin puu tietueen alkuun, eli ensimmäiseen solmuun.
<i>iArvo</i>	Arvo, jota etsitään syvyysshaussa.

Returns

int Palauttaa arvon 0 tai 1, riippuen siitä, löytyykö etsitty arvo.

Definition at line 17 of file haut.c.

```

17     {
18         // Syvyyshaku arvon mukaan, kirjoitetaan käydyt solmut tiedostoon.
19         int iLoytyi = 0;
20
21         if (pAlku == NULL) {
22             return 0;
23         }

```

```

24
25     if (pAlku != NULL) {
26         kirjoitaTiedostoon(pNimi, pAlku);
27
28         if (pAlku->iArvo == iArvo) {
29             iLoytyi = 1;
30             printf("Arvo %d löytyi.\n", iArvo);
31         } else {
32             if (iLoytyi == 0) {
33                 iLoytyi = syvyyshaku(pNimi, pAlku->pVasen, iArvo);
34             }
35             if (iLoytyi == 0) {
36                 iLoytyi = syvyyshaku(pNimi, pAlku->pOikea, iArvo);
37             }
38         }
39     }
40     return (iLoytyi);
41 }

```

4.4.2.4 tarkistaLoytzykoSyvyyshauulla()

```

void tarkistaLoytzykoSyvyyshauulla (
    char * aNimiKirjoitettava,
    PUU * pJuuriSolmu,
    int iArvo )

```

Tarkistaa, löytyykö syvyysaussa etsitty arvo. Jos arvoa ei löydy, tulostaa tiedon siitä.

Parameters

<i>aNimiKirjoitettava</i>	Kayttajan antama kirjoitettavan tiedoston nimi.
<i>pJuuriSolmu</i>	Osoitin puu tietueen alkuun, eli ensimmäiseen solmuun.
<i>iArvo</i>	Syvyysaussa haettava numeroarvo.

Returns

void

Definition at line 54 of file haut.c.

```

54
55     int iLoytyi = 0;
56     iLoytyi = syvyyshaku(aNimiKirjoitettava, pJuuriSolmu, iArvo);
57     if (iLoytyi == 0) {
58         printf("Arvoa %d ei löytynyt puusta.\n", iArvo);
59     }
60 }

```

4.4.2.5 vapautaMuistiJono()

```

JONO* vapautaMuistiJono (
    JONO * pAlku )

```

Vapauttaa jonon alkioita varten varatun muistin.

Parameters

<i>pAlku</i>	Osoitin jonon ensimmäiseen alkioon.
--------------	-------------------------------------

Returns

pAlku Palauttaa NULL, koska jono on tyhjä.

Definition at line 151 of file haut.c.

```

151                                     {
152     // Jonon muistin vapauttaminen.
153     JONO *ptr = pAlku;
154     JONO *pSeuraava = NULL;
155
156     while (ptr != NULL) {
157         pSeuraava = ptr->pSeuraava;
158         free(ptr);
159         ptr = pSeuraava;
160     }
161     pAlku = NULL;
162     return (pAlku);
163 }
```

4.4.2.6 varaaMuistiaJonolle()

```

JONO* varaaMuistiaJonolle (
    PUU * pSolmu )
```

Varaa muistia jonon uudelle alkiolle leveyshakua varten.

Parameters

<i>pSolmu</i>	Osoitin kasiteltavaan puun solmuun.
---------------	-------------------------------------

Returns

pUusi Osoitin uuteen jonon alkioon.

Definition at line 128 of file haut.c.

```

128                                     {
129     JONO *pUusi = NULL;
130
131     // Muistin varaaminen jonolle.
132     if ((pUusi = (JONO *)malloc(sizeof(JONO))) == NULL) {
133         perror("Muistin varaus epäonnistui, lopetetaan");
134         exit(0);
135     }
136
137     pUusi->pSolmu = pSolmu;
138     pUusi->pSeuraava = NULL;
139
140     return (pUusi);
141 }
```

4.5 linkitettylista.c File Reference

```

#include "linkitettylista.h"
#include <ctype.h>
```



```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

Functions

- **TIEDOT * lue** (char *pNimi, **TIEDOT *pAlku**)
Lukee tiedoston.
- **TIEDOT * varaaMuistia** (**TIEDOT *pAlku**, char *pNimi, int iMaara)
Varaa muistin ja lisää uuden solmun kaksisuuntaisen linkitetyn listan.
- **TIEDOT * vapautaMuisti** (**TIEDOT *pAlku**)
Vapauttaa linkitetyn listan varaaman muistin.
- void **kirjoitaTiedostoAlustaLoppuun** (char *pNimi, **TIEDOT *pAlku**)
Kirjoittaa linkitetyn listan tiedostoon alusta loppuun.
- void **kirjoitaTiedostoLopustaAlkuun** (char *pNimi, **TIEDOT *pAlku**)
Kirjoittaa linkitetyn listan tiedostoon lopusta alkuun.
- **TIEDOT * lisaaListaan** (**TIEDOT *pAlku**, int iIndeksi, char *pNimi, int iArvo)
Lisää linkitettyyn listaan käyttäjän syöttämät tiedot.
- **TIEDOT * poistaListastaAlkio** (**TIEDOT *pAlku**, int iLuvuVaiNimi, char *pTieto)
Poistaa linkitetystä listasta alkion.
- **TIEDOT * poistaListastaLkmPeruusteella** (**TIEDOT *pAlku**, int iLKM)
Poistetaan linkitetystä listasta alkio lukumäärän perusteella.
- **TIEDOT * poistaListastaNimenPerusteella** (**TIEDOT *pAlku**, char *pNimi)
Poistetaan linkitetystä listasta alkio nimen perusteella.
- int **useammallaAlkiollaSamaLKM** (**TIEDOT *pAlku**, int iLKM)
Etsii kaikki alkiot, joiden nimien lukumäärä on käyttäjän antama luku.
- int **onkoLukuVaiNimi** (char *Tieto)
Selvittää onko käyttäjän antama syöte luku vai nimi.
- int **laskeListanPituus** (**TIEDOT *pAlku**)
Laskee linkitetyn listan pituuden.
- **TIEDOT * halkaise** (**TIEDOT *pAlku**)
Halkaisee linkitetyn listan kahteen puoliskoon.
- **TIEDOT * lomit** (**TIEDOT *p1**, **TIEDOT *p2**)
Lomittaa (merge) kaksi lajiteltua listaa yhdeksi.
- **TIEDOT * lomitLajittelu** (**TIEDOT *pAlku**)
Lajittelee listan rekursiivisella lomitlajittelulla (Merge sort).
- **TIEDOT * lisaysLajittelu** (**TIEDOT *pAlku**)
Lajittelee listan lisayslajittelulla (Insertion sort).

4.5.1 Function Documentation

4.5.1.1 halkaise()

```
TIEDOT* halkaise (
    TIEDOT * pAlku )
```

Parameters

<i>pAlku</i>	Osoitin linkitetyn listan alkuun.
--------------	-----------------------------------

Returns

TIEDOT* Osoitin listan jälkimmäisen puoliskon alkuun.

Definition at line 434 of file linkitettylista.c.

```

434     {
435         TIEDOT *p1 = NULL, *p2 = NULL;
436         int i;
437         int iPituus = 0;
438         int iKeskipiste = 0;
439
440         iPituus = laskeListanPituus(pAlku);
441         iKeskipiste = iPituus / 2 - 1;
442
443         p1 = pAlku;
444         for (i = 0; i < iKeskipiste; i++) {
445             p1 = p1->pSeuraava;
446         }
447
448         // Halkaistaan lista kahtia, p1 päättyy siihen mistä p2 alkaa
449         p2 = p1->pSeuraava;
450         p1->pSeuraava = NULL;
451         if (p2 != NULL) {
452             p2->pEdellinen = NULL;
453         }
454
455         // palautetaan osoitin toisen listan alkuun
456         return p2;
457     }
```

4.5.1.2 kirjoitaTiedostoAlustaLoppuun()

```

void kirjoitaTiedostoAlustaLoppuun (
    char * pNimi,
    TIEDOT * pAlku )
```

Parameters

<i>pNimi</i>	Kirjoitettavan tiedoston nimi.
<i>pAlku</i>	Osoitin linkitetyn listan alkuun.

Returns

void

Definition at line 130 of file linkitettylista.c.

```

130     {
131         FILE *Tiedosto = NULL;
132         TIEDOT *ptr = NULL;
133
134         if (pAlku == NULL) {
135             printf("Lista on tyhjä, lue tiedosto ennen kirjoittamista.\n");
136             return;
137         }
138
139         // Kirjoitus tiedoston avaaminen
140         if ((Tiedosto = fopen(pNimi, "w")) == NULL) {
141             perror("Tiedoston avaaminen epäonnistui, lopetetaan");
```

```

142         exit(0);
143     }
144
145     // Tiedostoon kirjoittaminen
146     ptr = pAlku;
147     while (ptr != NULL) {
148         fprintf(Tiedosto, "%s,%d\n", ptr->aSukunimi, ptr->iYhteensa);
149         ptr = ptr->pSeuraava;
150     }
151
152     // Tiedoston sulkeminen
153     fclose(Tiedosto);
154     printf("Tiedosto %s kirjoitettu.\n", pNimi);
155     return;
156 }

```

4.5.1.3 kirjoitaTiedostoLopustaAlkuun()

```

void kirjoitaTiedostoLopustaAlkuun (
    char * pNimi,
    TIEDOT * pAlku )

```

Parameters

<i>pNimi</i>	Kirjoitettavan tiedoston nimi.
<i>pAlku</i>	Osoitin linkitetyn listan alkuun.

Returns

void

Definition at line 165 of file linkitettylista.c.

```

165                                     {
166     FILE *Tiedosto = NULL;
167     TIEDOT *ptr = pAlku;
168
169     if (pAlku == NULL) {
170         printf("Lista on tyhjä, lue tiedosto ennen kirjoittamista.\n");
171         return;
172     }
173
174     if ((Tiedosto = fopen(pNimi, "w")) == NULL) {
175         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
176         exit(0);
177     }
178
179     // Mennään listan loppuun.
180     while (ptr->pSeuraava != NULL) {
181         ptr = ptr->pSeuraava;
182     }
183
184     // Lopusta alkuun, kirjoitetaan tiedostoon.
185     while (ptr != NULL) {
186         fprintf(Tiedosto, "%s,%d\n", ptr->aSukunimi, ptr->iYhteensa);
187         ptr = ptr->pEdellinen;
188     }
189
190     fclose(Tiedosto);
191     printf("Tiedosto %s kirjoitettu.\n", pNimi);
192     return;
193 }

```

4.5.1.4 laskeListanPituus()

```

int laskeListanPituus (
    TIEDOT * pAlku )

```

Parameters

<i>pAlku</i>	Osoitin linkitetyn listan alkuun.
--------------	-----------------------------------

Returns

int Linkitetyn listan alkioden lukumäärä.

Definition at line 419 of file linkitettylista.c.

```

419                                     {
420     int i = 0;
421     while (pAlku != NULL) {
422         i++;
423         pAlku = pAlku->pSeuraava;
424     }
425     return i;
426 }
```

4.5.1.5 lisääListaan()

```

TIEDOT* lisääListaan (
    TIEDOT * pAlku,
    int iIndeksi,
    char * pNimi,
    int iArvo )
```

Lisaa käyttäjän haluamaan kohtaan (indeksi) tiedot lisattavasta nimestä ja niiden lukumaarasta.

Parameters

<i>pAlku</i>	Osoitin linkitetyn listan alkuun.
<i>iIndeksi</i>	Käyttäjän antama indeksi, eli kohta, johon tietue lisataan linkitettyssä listassa. Jos lista on tyhjä, lisataan automaattisesti ensimmäiseksi elementiksi. Jos indeksi on suurempi kuin listan elementtien määrä, lisataan automaattisesti viimeiseksi.
<i>pNimi</i>	Lisattava nimi.
<i>iArvo</i>	Lisattavan nimen lukumaara.

Returns

pAlku Osoitin listan nykyiseen alkuun.

Definition at line 208 of file linkitettylista.c.

```

208                                     {
209     TIEDOT *pUusi = NULL;
210     TIEDOT *ptr = pAlku;
211     int i = 1;
212
213     // Varataan muistia uudelle linkitetyn listan solmulle.
214     if ((pUusi = (TIEDOT *)malloc(sizeof(TIEDOT))) == NULL) {
215         perror("Muistin varaus epäonnistui, lopetetaan");
216         exit(0);
217     }
218
219     // Lisataan tiedot tietueeseen.
220     strcpy(pUusi->aSukunimi, pNimi);
221     pUusi->iYhteensa = iArvo;
222 }
```

```

223 // Jos lista on tyhja, lisataan uusi solmu ensimmäiseksi.
224 if (pAlku == NULL) {
225     pUusi->pEdellinen = NULL;
226     pUusi->pSeuraava = NULL;
227     pAlku = pUusi;
228     return (pAlku);
229 } else {
230     // Jos lisataan listan alkuun, eli indeksi = 0.
231     if (iIndeksi <= 1) {
232         pUusi->pEdellinen = NULL;
233         pUusi->pSeuraava = pAlku;
234         pAlku->pEdellinen = pUusi;
235         pAlku = pUusi;
236         return (pAlku);
237     }
238
239     // Etsitaan listalta oikea kohta, johon tiedot lisataan.
240     while (ptr->pSeuraava != NULL && i < (iIndeksi - 1)) {
241         ptr = ptr->pSeuraava;
242         i++;
243     }
244     // Lisataan tiedot oikeaan kohtaan linkitettyä listaa.
245     // Paivitetään osoittimet osoittamaan oikeisiin kohtiin.
246     pUusi->pSeuraava = ptr->pSeuraava;
247     pUusi->pEdellinen = ptr;
248
249     if (ptr->pSeuraava != NULL) {
250         ptr->pSeuraava->pEdellinen = pUusi;
251     }
252
253     ptr->pSeuraava = pUusi;
254 }
255 return (pAlku);
256 }

```

4.5.1.6 lisaysLajittelu()

```

TIEDOT* lisaysLajittelu (
    TIEDOT * pAlku )

```

Parameters

<i>pAlku</i>	Osoitin lajiteltavan linkitetyn listan alkuun.
--------------	--

Returns

TIEDOT* Lajiteltu lista.

Definition at line 587 of file linkitettylista.c.

```

587 {
588     // Jos lista on tyhjä
589     if (pAlku == NULL) {
590         return pAlku;
591     }
592
593     TIEDOT *pLajittelematon = pAlku; // Osoitin lajiteltavan listan nykyiseen solmuun
594     TIEDOT *pLajiteltu = NULL;       // Tehdään toinen lista, johon lajitellut tiedot laitetaan
595     TIEDOT *pSeuraava = NULL;
596     TIEDOT *ptr = NULL; // Osoitin, jolla läpikäydään lajiteltua listaa.
597
598     /* Käydään lajittelematonta listaa läpi ja lisätään alkiot
599     * lajiteltuun listaan pienenevässä järjestyksessä arvon mukaan.
600     */
601     while (pLajittelematon != NULL) {
602         pSeuraava = pLajittelematon->pSeuraava; // pSeuraava talteen ennen listan läpikäyntiä
603
604         // irrotetaan alkio
605         if (pSeuraava != NULL) {
606             pSeuraava->pEdellinen = pLajittelematon->pEdellinen;
607         }
608         if (pLajittelematon->pEdellinen != NULL) {

```

```

609         pLajittelematon->pEdellinen->pSeuraava = pSeuraava;
610     }
611     pLajittelematon->pSeuraava = NULL;
612     pLajittelematon->pEdellinen = NULL;
613
614     if (pLajiteltu == NULL || pLajiteltu->iYhteensa < pLajittelematon->iYhteensa ||
615         (pLajiteltu->iYhteensa == pLajittelematon->iYhteensa &&
616          strcmp(pLajiteltu->aSukunimi, pLajittelematon->aSukunimi) < 0)) {
617         // Lisätään lajitellun listan alkuun
618         pLajittelematon->pSeuraava = pLajiteltu;
619         if (pLajiteltu != NULL) {
620             pLajiteltu->pEdellinen = pLajittelematon;
621         }
622         pLajiteltu = pLajittelematon;
623
624     } else {
625         ptr = pLajiteltu;
626         while (ptr->pSeuraava != NULL &&
627              (ptr->pSeuraava->iYhteensa > pLajittelematon->iYhteensa ||
628               (ptr->pSeuraava->iYhteensa == pLajittelematon->iYhteensa &&
629                strcmp(ptr->pSeuraava->aSukunimi, pLajittelematon->aSukunimi) > 0))) {
630             ptr = ptr->pSeuraava;
631         }
632         pLajittelematon->pSeuraava = ptr->pSeuraava;
633         if (ptr->pSeuraava != NULL) {
634             ptr->pSeuraava->pEdellinen = pLajittelematon;
635         }
636         ptr->pSeuraava = pLajittelematon;
637         pLajittelematon->pEdellinen = ptr;
638     }
639     pLajittelematon = pSeuraava;
640 }
641 return pLajiteltu;
642 }

```

4.5.1.7 lomitus()

```

TIEDOT* lomitus (
    TIEDOT * p1,
    TIEDOT * p2 )

```

Parameters

<i>p1</i>	Osoitin ensimmäisen lajitellun listan alkuun.
<i>p2</i>	Osoitin jälkimmäisen lajitellun listan alkuun.

Returns

TIEDOT* yhdistetty lajiteltu lista.

Definition at line 466 of file linkitettylista.c.

```

466     {
467         // Jos kumpikaan lista on tyhjä, palautetaan toinen.
468         if (p1 == NULL)
469             return p2;
470         if (p2 == NULL)
471             return p1;
472
473         TIEDOT *p3 = NULL; // Uuden listan alku.
474         TIEDOT *p3Loppu = NULL; // Uuden lista loppu, jotta loppuun lisääminen onnistuu helposti.
475
476         // Käydään molempia listoja läpi:
477         while (p1 != NULL && p2 != NULL) {
478             int iVertailu =
479                 strcmp(p1->aSukunimi, p2->aSukunimi); // Jos sama arvo, lisätään aakkosjärjestyksessä.
480             if (p1->iYhteensa > p2->iYhteensa || (p1->iYhteensa == p2->iYhteensa && iVertailu > 0)) {
481                 TIEDOT *ptr = p2; // p2[0] talteen
482                 // Poistetaan p2[0] listasta.
483                 p2 = p2->pSeuraava;

```

```

484         if (p2 != NULL) {
485             p2->pEdellinen = NULL;
486         }
487         // Lisätään p2[0] uuden listan loppuun.
488         ptr->pSeuraava = NULL;
489         ptr->pEdellinen = p3Loppu;
490         if (p3Loppu != NULL) {
491             p3Loppu->pSeuraava = ptr;
492         } else {
493             p3 = ptr; // ensimmäinen alkio, jos lista tyhjä
494         }
495         p3Loppu = ptr; // päivitetään loppu.
496     } else {
497         TIEDOT *ptr = p1; // p1[0] talteen
498         p1 = p1->pSeuraava;
499         // Poistetaan p1[0] listasta.
500         if (p1 != NULL) {
501             p1->pEdellinen = NULL;
502         }
503         // Lisätään p1[0] uuden listan loppuun.
504         ptr->pSeuraava = NULL;
505         ptr->pEdellinen = p3Loppu;
506         if (p3Loppu != NULL) {
507             p3Loppu->pSeuraava = ptr;
508         } else {
509             p3 = ptr; // ensimmäinen alkio, jos lista tyhjä
510         }
511         p3Loppu = ptr; // päivitetään loppu.
512     }
513 }
514
515 // Tässä kohtaa joko p1 tai p2 on tyhjä.
516
517 // Lisätään loput p1:stä uuden listan loppuun.
518 while (p1 != NULL) {
519     TIEDOT *ptr = p1;
520     p1 = p1->pSeuraava;
521     if (p1 != NULL) {
522         p1->pEdellinen = NULL;
523     }
524     ptr->pSeuraava = NULL;
525     ptr->pEdellinen = p3Loppu;
526     if (p3Loppu != NULL) {
527         p3Loppu->pSeuraava = ptr;
528     } else {
529         p3 = ptr;
530     }
531     p3Loppu = ptr;
532 }
533
534 // Lisätään loput p2:stä uuden listan loppuun.
535 while (p2 != NULL) {
536     TIEDOT *ptr = p2;
537     p2 = p2->pSeuraava;
538     if (p2 != NULL) {
539         p2->pEdellinen = NULL;
540     }
541     ptr->pSeuraava = NULL;
542     ptr->pEdellinen = p3Loppu;
543     if (p3Loppu != NULL) {
544         p3Loppu->pSeuraava = ptr;
545     } else {
546         p3 = ptr;
547     }
548     p3Loppu = ptr;
549 }
550 return p3;
551 }

```

4.5.1.8 lomitusLajittelu()

```

TIEDOT* lomitusLajittelu (
    TIEDOT * pAlku )

```

Parameters

<i>pAlku</i>	Osoitin lajiteltavan linkitetyn listan alkuun.
--------------	--

Returns

TIEDOT* Lajiteltu lista.

Definition at line 559 of file linkitettylista.c.

```

559                                     {
560     TIEDOT *p1 = NULL, *p2 = NULL;
561
562     // Jos lista on tyhjä tai sisältää vain yhden alkion
563     if (pAlku == NULL || pAlku->pSeuraava == NULL) {
564         return pAlku;
565     }
566
567     // Jaetaan lista kahtia
568     p2 = halkaise(pAlku);
569     p1 = pAlku;
570
571     // Lajitellaan molemmat puolikkaat
572     p1 = lomituspLajittelu(p1);
573     p2 = lomituspLajittelu(p2);
574
575     // Yhdistetään lajitellut puolikkaat
576     return lomitusp(p1, p2);
577 }
```

4.5.1.9 lue()

```

TIEDOT* lue (
    char * pNimi,
    TIEDOT * pAlku )
```

Avaa ja lukee käyttäjän syötteen mukaisen tiedoston. Kutsuu muistinvaraus aliohjelman muistin varaamiseksi.

Parameters

<i>pNimi</i>	Luettavan tiedoston nimi.
<i>pAlku</i>	Osoitin linkitetyn listan alkuun.

Returns

pAlku Uusi osoitin linkitetyn listan alkuun.

Definition at line 19 of file linkitettylista.c.

```

19                                     {
20     FILE *Tiedosto = NULL;
21     char aRivi[LEN] = "";
22     char *p1 = NULL, *p2 = NULL;
23     int iOtsikko = 0;
24     int iMaara = 0;
25
26     /* Tiedoston avaaminen. */
27
28     if ((Tiedosto = fopen(pNimi, "r")) == NULL) {
29         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
30         exit(0);
31     }
32
33     /* Tiedoston lukeminen */
34
35     while ((fgets(aRivi, LEN, Tiedosto)) != NULL) {
36
37         aRivi[strlen(aRivi) - 1] = '\0';
38
39         if (iOtsikko == 0) {
40             iOtsikko = 1;
41             continue;

```



```

42     }
43
44     if ((p1 = strtok(aRivi, ";")) == NULL) {
45         perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
46         exit(0);
47     }
48
49     if ((p2 = strtok(NULL, ";")) == NULL) {
50         perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
51         exit(0);
52     }
53
54     iMaara = atoi(p2);
55
56     /* Muistin varaaminen ja linkitetyn listan luominen. */
57
58     pAlku = varaaMuistia(pAlku, p1, iMaara);
59 }
60 /* Tiedoston sulkeminen. */
61 fclose(Tiedosto);
62 printf("Tiedosto '%s' luettu.\n", pNimi);
63 return (pAlku);
64 }

```

4.5.1.10 onkoLukuVaiNimi()

```

int onkoLukuVaiNimi (
    char * Tieto )

```

Parameters

<i>Tieto</i>	Käyttäjän antama syöte.
--------------	-------------------------

Returns

int *Tieto*, onko luku vai nimi, jos -2, niin käyttäjä antoi luvun, jos joku muu niin se on luku.

Definition at line 388 of file linkitettylista.c.

```

388     {
389         int iVastaus = 0;
390         int i = 0;
391         int iPituus = 0;
392
393         iPituus = strlen(Tieto);
394
395         for (i = 0; i < iPituus; i++) {
396             if (isdigit(Tieto[i])) {
397                 iVastaus = -1;
398             } else {
399                 iVastaus = -2;
400                 break;
401             }
402         }
403
404         if (iVastaus == -1) {
405             iVastaus = atoi(Tieto);
406         }
407
408         return (iVastaus);
409 }

```

4.5.1.11 poistaListastaAlkio()

```

TIEDOT* poistaListastaAlkio (
    TIEDOT * pAlku,
    int iLuvuVaiNimi,
    char * pTieto )

```

Parameters

<i>pAlku</i>	Osoitin linkitetyn listan alkuun.
<i>iLuvuVaiNimi</i>	Tieto siitä, haluaako käyttäjä poistaa alkion nimen vai luvun perusteella.
<i>pTieto</i>	Poistettavan alkion tieto, nimi tai luku.

Returns

TIEDOT* pAlku Osoitin listan nykyiseen alkuun.

Definition at line 266 of file linkitettylista.c.

```

266                                     {
267     int iSamojenAlkiodenLkm = 0;
268     char aPoistettavaNimi[LEN] = "";
269
270     if (iLuvuVaiNimi == -2) {
271         pAlku = poistaListastaNimenPerusteella(pAlku, pTieto);
272     } else {
273         iSamojenAlkiodenLkm = useammallaAlkiollaSamaLKM(pAlku, iLuvuVaiNimi);
274
275         if (iSamojenAlkiodenLkm > 1) {
276             kysyNimi("Anna nimi, joka poistetaan: ", aPoistettavaNimi);
277             pAlku = poistaListastaNimenPerusteella(pAlku, aPoistettavaNimi);
278         } else {
279             pAlku = poistaListastaLkmPeruusteella(pAlku, iLuvuVaiNimi);
280         }
281     }
282
283     return (pAlku);
284 }
```

4.5.1.12 poistaListastaLkmPeruusteella()

```

TIEDOT* poistaListastaLkmPeruusteella (
    TIEDOT * pAlku,
    int iLKM )
```

Käyttäjä on antanut lukumäärän. Alkio, jossa on tämä lukumäärä poistetaan.

Parameters

<i>pAlku</i>	Osoitin linkitetyn listan alkuun.
<i>iLKM</i>	Käyttäjän antama lukumäärä. Alkio, jossa on tämä lukumäärä poistetaan.

Returns

TIEDOT* Palautetaan linkitettylista, josta on poistettu alkio.

Definition at line 293 of file linkitettylista.c.

```

293                                     {
294     TIEDOT *ptr = pAlku;
295     TIEDOT *pEdellinen = NULL;
296
297     // Tarkistetaan, onko poistettava alkio ensimmäinen.
298     if ((ptr != NULL) && (ptr->iYhteensa == iLKM)) {
299         // Sijoitetaan osoitin osoittamaan seuraavaan alkioon.
300         pAlku = ptr->pSeuraava;
301         free(ptr);
```

```

302     printf("Alkio poistettu.\n");
303
304     // Jos poistettava alkio ei ole ensimmäinen.
305 } else {
306     // Etsitään kohtaa, jossa poistettava alkio on.
307     while (ptr != NULL) {
308         if (ptr->iYhteensa == iLKM) {
309             pEdellinen->pSeuraava = ptr->pSeuraava;
310             free(ptr);
311             printf("Alkio poistettu.\n");
312             break;
313         }
314         pEdellinen = ptr;
315         ptr = ptr->pSeuraava;
316     }
317 }
318
319 return (pAlku);
320 }

```

4.5.1.13 poistaListastaNimenPerusteella()

```

TIEDOT* poistaListastaNimenPerusteella (
    TIEDOT * pAlku,
    char * pNimi )

```

Käyttäjä on antanut nimen. Alkio, jossa on tämä nimi poistetaan.

Parameters

<i>pAlku</i>	Osoitin linkitetyn listan alkuun.
<i>pNimi</i>	Käyttäjän antama nimi. Alkio, jossa on tämä nimi poistetaan.

Returns

TIEDOT* TIEDOT* Palautetaan linkitettylista, josta on poistettu alkio.

Definition at line 329 of file linkitettylista.c.

```

329
330     TIEDOT *ptr = pAlku;
331     TIEDOT *pEdellinen = NULL;
332
333     // Tarkistetaan, onko poistettava alkio ensimmäinen.
334     if ((ptr != NULL) && (strcmp(ptr->aSukunimi, pNimi) == 0)) {
335         // Sijoitetaan osoitin osoittamaan seuraavaan alkioon.
336         pAlku = ptr->pSeuraava;
337         free(ptr);
338         printf("Alkio poistettu.\n");
339
340         // Jos poistettava alkio ei ole ensimmäinen.
341     } else {
342         // Etsitään kohtaa, jossa poistettava alkio on.
343         while (ptr != NULL) {
344             if (strcmp(ptr->aSukunimi, pNimi) == 0) {
345                 pEdellinen->pSeuraava = ptr->pSeuraava;
346                 free(ptr);
347                 printf("Alkio poistettu.\n");
348                 break;
349             }
350             pEdellinen = ptr;
351             ptr = ptr->pSeuraava;
352         }
353     }
354     return (pAlku);
355 }

```

4.5.1.14 useammallaAlkiollaSamaLKM()

```
int useammallaAlkiollaSamaLKM (
    TIEDOT * pAlku,
    int iLKM )
```

Parameters

<i>pAlku</i>	Osoitin linkitetyn listan alkuun.
<i>iLKM</i>	Käyttäjän antama lukumäärä, joiden määrä selvitetään.

Returns

int Palauttaa luvun, jossa on kaikki alkiot, joissa on käyttäjän antama luku.

Definition at line 364 of file linkitettylista.c.

```
364                                     {
365     int iSamoja = 0;
366     TIEDOT *ptr = NULL;
367     ptr = pAlku;
368
369     // Käydään linkitettylista läpi
370     while (ptr != NULL) {
371         // Lisätään summaan yksi, jos linkitetyn listan alkion lukumäärä on sama kuin käyttäjän
372         // antama luku.
373         if (ptr->iYhteensa == iLKM) {
374             iSamoja++;
375         }
376         ptr = ptr->pSeuraava;
377     }
378
379     return (iSamoja);
380 }
```

4.5.1.15 vapautaMuisti()

```
TIEDOT* vapautaMuisti (
    TIEDOT * pAlku )
```

Parameters

<i>pAlku</i>	Osoitin linkitetyn listan alkuun.
--------------	-----------------------------------

Returns

pAlku Osoitin, joka on NULL, koska lista on tyhjä.

Definition at line 111 of file linkitettylista.c.

```
111                                     {
112     /* Muistin vapauttaminen. */
113     TIEDOT *ptr = pAlku;
114     while (ptr != NULL) {
115         pAlku = ptr->pSeuraava;
116         free(ptr);
117         ptr = pAlku;
118     }
119     pAlku = NULL;
120     return (pAlku);
121 }
```

4.5.1.16 varaaMuistia()

```
TIEDOT* varaaMuistia (
    TIEDOT * pAlku,
    char * pNimi,
    int iMaara )
```

Parameters

<i>pAlku</i>	Osoitin linkitetyn listan alkuun.
<i>pNimi</i>	Merkkijono solmun nimeksi.
<i>iMaara</i>	Luku, joka asetetaan solmuun maaraksi.

Returns

pAlku Uusi osoitin linkitetyn listan alkuun.

Definition at line 74 of file linkitettylista.c.

```
74                                     {
75     TIEDOT *pUusi = NULL;
76     TIEDOT *ptr = NULL;
77
78     /* Muistin varaaminen. */
79
80     if ((pUusi = (TIEDOT *)malloc(sizeof(TIEDOT))) == NULL) {
81         perror("Muistin varaus epäonnistui, lopetetaan");
82         exit(0);
83     }
84
85     /* Kaksisuuntaisen linkitetyn listan luominen. */
86
87     strcpy(pUusi->aSukunimi, pNimi);
88     pUusi->iYhteensa = iMaara;
89     pUusi->pSeuraava = NULL;
90
91     if (pAlku == NULL) {
92         pUusi->pEdellinen = NULL;
93         pAlku = pUusi;
94     } else {
95         ptr = pAlku;
96         while (ptr->pSeuraava != NULL) {
97             ptr = ptr->pSeuraava;
98         }
99         ptr->pSeuraava = pUusi;
100        pUusi->pEdellinen = ptr;
101    }
102    return (pAlku);
103 }
```

4.6 linkitettylista.h File Reference

```
#include "yleiset.h"
```

Classes

- struct [tiedot](#)

Typedefs

- typedef struct [tiedot](#) TIEDOT

Functions

- `TIEDOT * lue` (char *pNimi, `TIEDOT *pAlku`)
Lukee tiedoston.
- `TIEDOT * varaaMuistia` (`TIEDOT *pAlku`, char *pSukunimi, int iMaara)
Varaa muistin ja lisää uuden solmun kaksisuuntaisen linkitetyn listan.
- `TIEDOT * vapautaMuisti` (`TIEDOT *pAlku`)
Vapauttaa linkitetyn listan varaaman muistin.
- void `kirjoitaTiedostoAlustaLoppuun` (char *pKirjoitaTiedostoNimi, `TIEDOT *pAlku`)
Kirjoittaa linkitetyn listan tiedostoon alusta loppuun.
- void `kirjoitaTiedostoLopustaAlkuun` (char *pKirjoitaTiedostoNimi, `TIEDOT *pAlku`)
Kirjoittaa linkitetyn listan tiedostoon lopusta alkuun.
- `TIEDOT * lisaaListaan` (`TIEDOT *pAlku`, int iIndeksi, char *pNimi, int iArvo)
Lisää linkitettyyn listaan käyttäjän syöttämät tiedot.
- `TIEDOT * poistaListastaLkmPeruusteella` (`TIEDOT *pAlku`, int iLKM)
Poistetaan linkitetystä listasta alkio lukumäärän perusteella.
- int `useammallaAlkiollaSamaLKM` (`TIEDOT *pAlku`, int iLKM)
Etsii kaikki alkiot, joiden nimien lukumäärä on käyttäjän antama luku.
- `TIEDOT * poistaListastaNimenPerusteella` (`TIEDOT *pAlku`, char *pNimi)
Poistetaan linkitetystä listasta alkio nimen perusteella.
- int `laskeListanPituus` (`TIEDOT *pAlku`)
Laskee linkitetyn listan pituuden.
- `TIEDOT * halkaise` (`TIEDOT *pAlku`)
Halkaisee linkitetyn listan kahteen puoliskoon.
- `TIEDOT * lomit` (`TIEDOT *p1`, `TIEDOT *p2`)
Lomittaa (merge) kaksi lajiteltua listaa yhdeksi.
- `TIEDOT * lomitLajittelu` (`TIEDOT *pAlku`)
Lajittelee listan rekursiivisella lomitushajittelulla (Merge sort).
- `TIEDOT * lisaysLajittelu` (`TIEDOT *pAlku`)
Lajittelee listan lisayshajittelulla (Insertion sort).
- int `onkoLukuVaiNimi` (char *Tieto)
Selvittää onko käyttäjän antama syöte luku vai nimi.
- `TIEDOT * poistaListastaAlkio` (`TIEDOT *pAlku`, int iLuvuVaiNimi, char *pTieto)
Poistaa linkitetystä listasta alkion.

4.6.1 Typedef Documentation

4.6.1.1 TIEDOT

```
typedef struct tiedot TIEDOT
```

4.6.2 Function Documentation

4.6.2.1 halkaise()

```
TIEDOT* halkaise (
    TIEDOT * pAlku )
```

Parameters

<i>pAlku</i>	Osoitin linkitetyn listan alkuun.
--------------	-----------------------------------

Returns

TIEDOT* Osoitin listan jälkimmäisen puoliskon alkuun.

Definition at line 434 of file linkitettylista.c.

```

434     {
435     TIEDOT *p1 = NULL, *p2 = NULL;
436     int i;
437     int iPituus = 0;
438     int iKeskipiste = 0;
439
440     iPituus = laskeListanPituus(pAlku);
441     iKeskipiste = iPituus / 2 - 1;
442
443     p1 = pAlku;
444     for (i = 0; i < iKeskipiste; i++) {
445         p1 = p1->pSeuraava;
446     }
447
448     // Halkaistaan lista kahtia, p1 päättyy siihen mistä p2 alkaa
449     p2 = p1->pSeuraava;
450     p1->pSeuraava = NULL;
451     if (p2 != NULL) {
452         p2->pEdellinen = NULL;
453     }
454
455     // palautetaan osoitin toisen listan alkuun
456     return p2;
457 }
```

4.6.2.2 kirjoitaTiedostoAlustaLoppuun()

```

void kirjoitaTiedostoAlustaLoppuun (
    char * pNimi,
    TIEDOT * pAlku )
```

Parameters

<i>pNimi</i>	Kirjoitettavan tiedoston nimi.
<i>pAlku</i>	Osoitin linkitetyn listan alkuun.

Returns

void

Definition at line 130 of file linkitettylista.c.

```

130     {
131     FILE *Tiedosto = NULL;
132     TIEDOT *ptr = NULL;
133
134     if (pAlku == NULL) {
135         printf("Lista on tyhjä, lue tiedosto ennen kirjoittamista.\n");
136         return;
137     }
138
139     // Kirjoitus tiedoston avaaminen
140     if ((Tiedosto = fopen(pNimi, "w")) == NULL) {
141         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
```

```

142         exit(0);
143     }
144
145     // Tiedostoon kirjoittaminen
146     ptr = pAlku;
147     while (ptr != NULL) {
148         fprintf(Tiedosto, "%s,%d\n", ptr->aSukunimi, ptr->iYhteensa);
149         ptr = ptr->pSeuraava;
150     }
151
152     // Tiedoston sulkeminen
153     fclose(Tiedosto);
154     printf("Tiedosto %s kirjoitettu.\n", pNimi);
155     return;
156 }

```

4.6.2.3 kirjoitaTiedostoLopustaAlkuun()

```

void kirjoitaTiedostoLopustaAlkuun (
    char * pNimi,
    TIEDOT * pAlku )

```

Parameters

<i>pNimi</i>	Kirjoitettavan tiedoston nimi.
<i>pAlku</i>	Osoitin linkitetyn listan alkuun.

Returns

void

Definition at line 165 of file linkitettylista.c.

```

165     {
166         FILE *Tiedosto = NULL;
167         TIEDOT *ptr = pAlku;
168
169         if (pAlku == NULL) {
170             printf("Lista on tyhjä, lue tiedosto ennen kirjoittamista.\n");
171             return;
172         }
173
174         if ((Tiedosto = fopen(pNimi, "w")) == NULL) {
175             perror("Tiedoston avaaminen epäonnistui, lopetetaan");
176             exit(0);
177         }
178
179         // Mennään listan loppuun.
180         while (ptr->pSeuraava != NULL) {
181             ptr = ptr->pSeuraava;
182         }
183
184         // Lopusta alkuun, kirjoitetaan tiedostoon.
185         while (ptr != NULL) {
186             fprintf(Tiedosto, "%s,%d\n", ptr->aSukunimi, ptr->iYhteensa);
187             ptr = ptr->pEdellinen;
188         }
189
190         fclose(Tiedosto);
191         printf("Tiedosto %s kirjoitettu.\n", pNimi);
192         return;
193     }

```

4.6.2.4 laskeListanPituus()

```

int laskeListanPituus (
    TIEDOT * pAlku )

```


Parameters

<i>pAlku</i>	Osoitin linkitetyn listan alkuun.
--------------	-----------------------------------

Returns

int Linkitetyn listan alkioden lukumäärä.

Definition at line 419 of file linkitettylista.c.

```

419                                     {
420     int i = 0;
421     while (pAlku != NULL) {
422         i++;
423         pAlku = pAlku->pSeuraava;
424     }
425     return i;
426 }
```

4.6.2.5 lisaaListaan()

```

TIEDOT* lisaaListaan (
    TIEDOT * pAlku,
    int iIndeksi,
    char * pNimi,
    int iArvo )
```

Lisaa käyttäjän haluamaan kohtaan (indeksi) tiedot lisattavasta nimestä ja niiden lukumaarasta.

Parameters

<i>pAlku</i>	Osoitin linkitetyn listan alkuun.
<i>iIndeksi</i>	Käyttäjän antama indeksi, eli kohta, johon tietue lisataan linkitettyssä listassa. Jos lista on tyhjä, lisataan automaattisesti ensimmäiseksi elementiksi. Jos indeksi on suurempi kuin listan elementtien määrä, lisataan automaattisesti viimeiseksi.
<i>pNimi</i>	Lisattava nimi.
<i>iArvo</i>	Lisattavan nimen lukumaara.

Returns

pAlku Osoitin listan nykyiseen alkuun.

Definition at line 208 of file linkitettylista.c.

```

208                                     {
209     TIEDOT *pUusi = NULL;
210     TIEDOT *ptr = pAlku;
211     int i = 1;
212
213     // Varataan muistia uudelle linkitetyn listan solmulle.
214     if ((pUusi = (TIEDOT *)malloc(sizeof(TIEDOT))) == NULL) {
215         perror("Muistin varaus epäonnistui, lopetetaan");
216         exit(0);
217     }
218
219     // Lisataan tiedot tietueeseen.
220     strcpy(pUusi->aSukunimi, pNimi);
221     pUusi->iYhteensa = iArvo;
222 }
```

```

223 // Jos lista on tyhja, lisataan uusi solmu ensimmäiseksi.
224 if (pAlku == NULL) {
225     pUusi->pEdellinen = NULL;
226     pUusi->pSeuraava = NULL;
227     pAlku = pUusi;
228     return (pAlku);
229 } else {
230     // Jos lisataan listan alkuun, eli indeksi = 0.
231     if (iIndeksi <= 1) {
232         pUusi->pEdellinen = NULL;
233         pUusi->pSeuraava = pAlku;
234         pAlku->pEdellinen = pUusi;
235         pAlku = pUusi;
236         return (pAlku);
237     }
238
239     // Etsitaan listalta oikea kohta, johon tiedot lisataan.
240     while (ptr->pSeuraava != NULL && i < (iIndeksi - 1)) {
241         ptr = ptr->pSeuraava;
242         i++;
243     }
244     // Lisataan tiedot oikeaan kohtaan linkitettyä listaa.
245     // Paivitetään osoittimet osoittamaan oikeisiin kohtiin.
246     pUusi->pSeuraava = ptr->pSeuraava;
247     pUusi->pEdellinen = ptr;
248
249     if (ptr->pSeuraava != NULL) {
250         ptr->pSeuraava->pEdellinen = pUusi;
251     }
252
253     ptr->pSeuraava = pUusi;
254 }
255 return (pAlku);
256 }

```

4.6.2.6 lisaysLajittelu()

```

TIEDOT* lisaysLajittelu (
    TIEDOT * pAlku )

```

Parameters

<i>pAlku</i>	Osoitin lajiteltavan linkitetyn listan alkuun.
--------------	--

Returns

TIEDOT* Lajiteltu lista.

Definition at line 587 of file linkitettylista.c.

```

587 {
588     // Jos lista on tyhjä
589     if (pAlku == NULL) {
590         return pAlku;
591     }
592
593     TIEDOT *pLajittelematon = pAlku; // Osoitin lajiteltavan listan nykyiseen solmuun
594     TIEDOT *pLajiteltu = NULL;      // Tehdään toinen lista, johon lajitellut tiedot laitetaan
595     TIEDOT *pSeuraava = NULL;
596     TIEDOT *ptr = NULL; // Osoitin, jolla läpikäydään lajiteltua listaa.
597
598     /* Käydään lajittelematonta listaa läpi ja lisätään alkiot
599     * lajiteltuun listaan pienenevässä järjestyksessä arvon mukaan.
600     */
601     while (pLajittelematon != NULL) {
602         pSeuraava = pLajittelematon->pSeuraava; // pSeuraava talteen ennen listan läpikäyntiä
603
604         // irrotetaan alkio
605         if (pSeuraava != NULL) {
606             pSeuraava->pEdellinen = pLajittelematon->pEdellinen;
607         }
608         if (pLajittelematon->pEdellinen != NULL) {

```

```

609     pLajittelematon->pEdellinen->pSeuraava = pSeuraava;
610 }
611 pLajittelematon->pSeuraava = NULL;
612 pLajittelematon->pEdellinen = NULL;
613
614 if (pLajiteltu == NULL || pLajiteltu->iYhteensa < pLajittelematon->iYhteensa ||
615     (pLajiteltu->iYhteensa == pLajittelematon->iYhteensa &&
616      strcmp(pLajiteltu->aSukunimi, pLajittelematon->aSukunimi) < 0)) {
617     // Lisätään lajitellun listan alkuun
618     pLajittelematon->pSeuraava = pLajiteltu;
619     if (pLajiteltu != NULL) {
620         pLajiteltu->pEdellinen = pLajittelematon;
621     }
622     pLajiteltu = pLajittelematon;
623
624 } else {
625     ptr = pLajiteltu;
626     while (ptr->pSeuraava != NULL &&
627           (ptr->pSeuraava->iYhteensa > pLajittelematon->iYhteensa ||
628            (ptr->pSeuraava->iYhteensa == pLajittelematon->iYhteensa &&
629             strcmp(ptr->pSeuraava->aSukunimi, pLajittelematon->aSukunimi) > 0))) {
630         ptr = ptr->pSeuraava;
631     }
632     pLajittelematon->pSeuraava = ptr->pSeuraava;
633     if (ptr->pSeuraava != NULL) {
634         ptr->pSeuraava->pEdellinen = pLajittelematon;
635     }
636     ptr->pSeuraava = pLajittelematon;
637     pLajittelematon->pEdellinen = ptr;
638 }
639 pLajittelematon = pSeuraava;
640 }
641 return pLajiteltu;
642 }

```

4.6.2.7 lomitus()

```

TIEDOT* lomitus (
    TIEDOT * p1,
    TIEDOT * p2 )

```

Parameters

<i>p1</i>	Osoitin ensimmäisen lajitellun listan alkuun.
<i>p2</i>	Osoitin jälkimmäisen lajitellun listan alkuun.

Returns

TIEDOT* yhdistetty lajiteltu lista.

Definition at line 466 of file linkitettylista.c.

```

466 {
467     // Jos kumpikaan lista on tyhjä, palautetaan toinen.
468     if (p1 == NULL)
469         return p2;
470     if (p2 == NULL)
471         return p1;
472
473     TIEDOT *p3 = NULL; // Uuden listan alku.
474     TIEDOT *p3Loppu = NULL; // Uuden lista loppu, jotta loppuun lisääminen onnistuu helposti.
475
476     // Käydään molempia listoja läpi:
477     while (p1 != NULL && p2 != NULL) {
478         int iVertailu =
479             strcmp(p1->aSukunimi, p2->aSukunimi); // Jos sama arvo, lisätään aakkosjärjestyksessä.
480         if (p1->iYhteensa > p2->iYhteensa || (p1->iYhteensa == p2->iYhteensa && iVertailu > 0)) {
481             TIEDOT *ptr = p2; // p2[0] talteen
482             // Poistetaan p2[0] listasta.
483             p2 = p2->pSeuraava;

```

```

484         if (p2 != NULL) {
485             p2->pEdellinen = NULL;
486         }
487         // Lisätään p2[0] uuden listan loppuun.
488         ptr->pSeuraava = NULL;
489         ptr->pEdellinen = p3Loppu;
490         if (p3Loppu != NULL) {
491             p3Loppu->pSeuraava = ptr;
492         } else {
493             p3 = ptr; // ensimmäinen alkio, jos lista tyhjä
494         }
495         p3Loppu = ptr; // päivitetään loppu.
496     } else {
497         TIEDOT *ptr = p1; // p1[0] talteen
498         p1 = p1->pSeuraava;
499         // Poistetaan p1[0] listasta.
500         if (p1 != NULL) {
501             p1->pEdellinen = NULL;
502         }
503         // Lisätään p1[0] uuden listan loppuun.
504         ptr->pSeuraava = NULL;
505         ptr->pEdellinen = p3Loppu;
506         if (p3Loppu != NULL) {
507             p3Loppu->pSeuraava = ptr;
508         } else {
509             p3 = ptr; // ensimmäinen alkio, jos lista tyhjä
510         }
511         p3Loppu = ptr; // päivitetään loppu.
512     }
513 }
514
515 // Tässä kohtaa joko p1 tai p2 on tyhjä.
516
517 // Lisätään loput p1:stä uuden listan loppuun.
518 while (p1 != NULL) {
519     TIEDOT *ptr = p1;
520     p1 = p1->pSeuraava;
521     if (p1 != NULL) {
522         p1->pEdellinen = NULL;
523     }
524     ptr->pSeuraava = NULL;
525     ptr->pEdellinen = p3Loppu;
526     if (p3Loppu != NULL) {
527         p3Loppu->pSeuraava = ptr;
528     } else {
529         p3 = ptr;
530     }
531     p3Loppu = ptr;
532 }
533
534 // Lisätään loput p2:stä uuden listan loppuun.
535 while (p2 != NULL) {
536     TIEDOT *ptr = p2;
537     p2 = p2->pSeuraava;
538     if (p2 != NULL) {
539         p2->pEdellinen = NULL;
540     }
541     ptr->pSeuraava = NULL;
542     ptr->pEdellinen = p3Loppu;
543     if (p3Loppu != NULL) {
544         p3Loppu->pSeuraava = ptr;
545     } else {
546         p3 = ptr;
547     }
548     p3Loppu = ptr;
549 }
550 return p3;
551 }

```

4.6.2.8 lomitusLajittelu()

```

TIEDOT* lomitusLajittelu (
    TIEDOT * pAlku )

```

Parameters

<i>pAlku</i>	Osoitin lajiteltavan linkitetyn listan alkuun.
--------------	--

Returns

TIEDOT* Lajiteltu lista.

Definition at line 559 of file linkitettylista.c.

```

559                                     {
560     TIEDOT *p1 = NULL, *p2 = NULL;
561
562     // Jos lista on tyhjä tai sisältää vain yhden alkion
563     if (pAlku == NULL || pAlku->pSeuraava == NULL) {
564         return pAlku;
565     }
566
567     // Jaetaan lista kahtia
568     p2 = halkaise(pAlku);
569     p1 = pAlku;
570
571     // Lajitellaan molemmat puolikkaat
572     p1 = lomituspLajittelu(p1);
573     p2 = lomituspLajittelu(p2);
574
575     // Yhdistetään lajitellut puolikkaat
576     return lomitusp(p1, p2);
577 }
```

4.6.2.9 lue()

```

TIEDOT* lue (
    char * pNimi,
    TIEDOT * pAlku )
```

Avaa ja lukee käyttäjän syötteen mukaisen tiedoston. Kutsuu muistinvaraus aliohjelmää muistin varaamiseksi.

Parameters

<i>pNimi</i>	Luettavan tiedoston nimi.
<i>pAlku</i>	Osoitin linkitetyn listan alkuun.

Returns

pAlku Uusi osoitin linkitetyn listan alkuun.

Definition at line 19 of file linkitettylista.c.

```

19                                     {
20     FILE *Tiedosto = NULL;
21     char aRivi[LEN] = "";
22     char *p1 = NULL, *p2 = NULL;
23     int iOtsikko = 0;
24     int iMaara = 0;
25
26     /* Tiedoston avaaminen. */
27
28     if ((Tiedosto = fopen(pNimi, "r")) == NULL) {
29         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
30         exit(0);
31     }
32
33     /* Tiedoston lukeminen */
34
35     while ((fgets(aRivi, LEN, Tiedosto)) != NULL) {
36
37         aRivi[strlen(aRivi) - 1] = '\0';
38
39         if (iOtsikko == 0) {
40             iOtsikko = 1;
41             continue;

```

```

42     }
43
44     if ((p1 = strtok(aRivi, ";")) == NULL) {
45         perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
46         exit(0);
47     }
48
49     if ((p2 = strtok(NULL, ";")) == NULL) {
50         perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
51         exit(0);
52     }
53
54     iMaara = atoi(p2);
55
56     /* Muistin varaaminen ja linkitetyn listan luominen. */
57
58     pAlku = varaaMuistia(pAlku, p1, iMaara);
59 }
60 /* Tiedoston sulkeminen. */
61 fclose(Tiedosto);
62 printf("Tiedosto '%s' luettu.\n", pNimi);
63 return (pAlku);
64 }

```

4.6.2.10 onkoLukuVaiNimi()

```

int onkoLukuVaiNimi (
    char * Tieto )

```

Parameters

<i>Tieto</i>	Käyttäjän antama syöte.
--------------	-------------------------

Returns

int Tieto, onko luku vai nimi, jos -2, niin käyttäjä antoi luvun, jos joku muu niin se on luku.

Definition at line 388 of file linkitettylista.c.

```

388     {
389         int iVastaus = 0;
390         int i = 0;
391         int iPituus = 0;
392
393         iPituus = strlen(Tieto);
394
395         for (i = 0; i < iPituus; i++) {
396             if (isdigit(Tieto[i])) {
397                 iVastaus = -1;
398             } else {
399                 iVastaus = -2;
400                 break;
401             }
402         }
403
404         if (iVastaus == -1) {
405             iVastaus = atoi(Tieto);
406         }
407
408         return (iVastaus);
409 }

```

4.6.2.11 poistaListastaAlkio()

```

TIEDOT* poistaListastaAlkio (
    TIEDOT * pAlku,
    int iLuvuVaiNimi,
    char * pTieto )

```

Parameters

<i>pAlku</i>	Osoitin linkitetyn listan alkuun.
<i>iLuvuVaiNimi</i>	Tieto siitä, haluaako käyttäjä poistaa alkion nimen vai luvun perusteella.
<i>pTieto</i>	Poistettavan alkion tieto, nimi tai luku.

Returns

TIEDOT* pAlku Osoitin listan nykyiseen alkuun.

Definition at line 266 of file linkitettylista.c.

```

266                                     {
267     int iSamojenAlkiodenLkm = 0;
268     char aPoistettavaNimi[LEN] = "";
269
270     if (iLuvuVaiNimi == -2) {
271         pAlku = poistaListastaNimenPerusteella(pAlku, pTieto);
272     } else {
273         iSamojenAlkiodenLkm = useamallaAlkiollaSamaLKM(pAlku, iLuvuVaiNimi);
274
275         if (iSamojenAlkiodenLkm > 1) {
276             kysyNimi("Anna nimi, joka poistetaan: ", aPoistettavaNimi);
277             pAlku = poistaListastaNimenPerusteella(pAlku, aPoistettavaNimi);
278         } else {
279             pAlku = poistaListastaLkmPeruusteella(pAlku, iLuvuVaiNimi);
280         }
281     }
282
283     return (pAlku);
284 }
```

4.6.2.12 poistaListastaLkmPeruusteella()

```

TIEDOT* poistaListastaLkmPeruusteella (
    TIEDOT * pAlku,
    int iLKM )
```

Käyttäjä on antanut lukumäärän. Alkio, jossa on tämä lukumäärä poistetaan.

Parameters

<i>pAlku</i>	Osoitin linkitetyn listan alkuun.
<i>iLKM</i>	Käyttäjän antama lukumäärä. Alkio, jossa on tämä lukumäärä poistetaan.

Returns

TIEDOT* Palautetaan linkitettylista, josta on poistettu alkio.

Definition at line 293 of file linkitettylista.c.

```

293                                     {
294     TIEDOT *ptr = pAlku;
295     TIEDOT *pEdellinen = NULL;
296
297     // Tarkistetaan, onko poistettava alkio ensimmäinen.
298     if ((ptr != NULL) && (ptr->iYhteensa == iLKM)) {
299         // Sijoitetaan osoitin osoittamaan seuraavaan alkioon.
300         pAlku = ptr->pSeuraava;
301         free(ptr);
```

```

302     printf("Alkio poistettu.\n");
303
304     // Jos poistettava alkio ei ole ensimmäinen.
305 } else {
306     // Etsitään kohtaa, jossa poistettava alkio on.
307     while (ptr != NULL) {
308         if (ptr->iYhteensa == iLKM) {
309             pEdellinen->pSeuraava = ptr->pSeuraava;
310             free(ptr);
311             printf("Alkio poistettu.\n");
312             break;
313         }
314         pEdellinen = ptr;
315         ptr = ptr->pSeuraava;
316     }
317 }
318
319 return (pAlku);
320 }

```

4.6.2.13 poistaListastaNimenPerusteella()

```

TIEDOT* poistaListastaNimenPerusteella (
    TIEDOT * pAlku,
    char * pNimi )

```

Käyttäjä on antanut nimen. Alkio, jossa on tämä nimi poistetaan.

Parameters

<i>pAlku</i>	Osoitin linkitetyn listan alkuun.
<i>pNimi</i>	Käyttäjän antama nimi. Alkio, jossa on tämä nimi poistetaan.

Returns

TIEDOT* TIEDOT* Palautetaan linkitettylista, josta on poistettu alkio.

Definition at line 329 of file linkitettylista.c.

```

329
330     TIEDOT *ptr = pAlku;
331     TIEDOT *pEdellinen = NULL;
332
333     // Tarkistetaan, onko poistettava alkio ensimmäinen.
334     if ((ptr != NULL) && (strcmp(ptr->aSukunimi, pNimi) == 0)) {
335         // Sijoitetaan osoitin osoittamaan seuraavaan alkioon.
336         pAlku = ptr->pSeuraava;
337         free(ptr);
338         printf("Alkio poistettu.\n");
339
340         // Jos poistettava alkio ei ole ensimmäinen.
341     } else {
342         // Etsitään kohtaa, jossa poistettava alkio on.
343         while (ptr != NULL) {
344             if (strcmp(ptr->aSukunimi, pNimi) == 0) {
345                 pEdellinen->pSeuraava = ptr->pSeuraava;
346                 free(ptr);
347                 printf("Alkio poistettu.\n");
348                 break;
349             }
350             pEdellinen = ptr;
351             ptr = ptr->pSeuraava;
352         }
353     }
354     return (pAlku);
355 }

```


4.6.2.14 useammallaAlkiollaSamaLKM()

```
int useammallaAlkiollaSamaLKM (
    TIEDOT * pAlku,
    int iLKM )
```

Parameters

<i>pAlku</i>	Osoitin linkitetyn listan alkuun.
<i>iLKM</i>	Käyttäjän antama lukumäärä, joiden määrä selvitetään.

Returns

int Palauttaa luvun, jossa on kaikki alkiot, joissa on käyttäjän antama luku.

Definition at line 364 of file linkitettylista.c.

```
364                                     {
365     int iSamoja = 0;
366     TIEDOT *ptr = NULL;
367     ptr = pAlku;
368
369     // Käydään linkitettylista läpi
370     while (ptr != NULL) {
371         // Lisätään summaan yksi, jos linkitetyn listan alkion lukumäärä on sama kuin käyttäjän
372         // antama luku.
373         if (ptr->iYhteensa == iLKM) {
374             iSamoja++;
375         }
376         ptr = ptr->pSeuraava;
377     }
378
379     return (iSamoja);
380 }
```

4.6.2.15 vapautaMuisti()

```
TIEDOT* vapautaMuisti (
    TIEDOT * pAlku )
```

Parameters

<i>pAlku</i>	Osoitin linkitetyn listan alkuun.
--------------	-----------------------------------

Returns

pAlku Osoitin, joka on NULL, koska lista on tyhjä.

Definition at line 111 of file linkitettylista.c.

```
111                                     {
112     /* Muistin vapauttaminen. */
113     TIEDOT *ptr = pAlku;
114     while (ptr != NULL) {
115         pAlku = ptr->pSeuraava;
116         free(ptr);
117         ptr = pAlku;
118     }
119     pAlku = NULL;
120     return (pAlku);
121 }
```

4.6.2.16 varaaMuistia()

```
TIEDOT* varaaMuistia (
    TIEDOT * pAlku,
    char * pNimi,
    int iMaara )
```

Parameters

<i>pAlku</i>	Osoitin linkitetyn listan alkuun.
<i>pNimi</i>	Merkkijono solmun nimeksi.
<i>iMaara</i>	Luku, joka asetetaan solmuun maaraksi.

Returns

pAlku Uusi osoitin linkitetyn listan alkuun.

Definition at line 74 of file linkitettylista.c.

```
74                                     {
75     TIEDOT *pUusi = NULL;
76     TIEDOT *ptr = NULL;
77
78     /* Muistin varaaminen. */
79
80     if ((pUusi = (TIEDOT *)malloc(sizeof(TIEDOT))) == NULL) {
81         perror("Muistin varaus epäonnistui, lopetetaan");
82         exit(0);
83     }
84
85     /* Kaksisuuntaisen linkitetyn listan luominen. */
86
87     strcpy(pUusi->aSukunimi, pNimi);
88     pUusi->iYhteensa = iMaara;
89     pUusi->pSeuraava = NULL;
90
91     if (pAlku == NULL) {
92         pUusi->pEdellinen = NULL;
93         pAlku = pUusi;
94     } else {
95         ptr = pAlku;
96         while (ptr->pSeuraava != NULL) {
97             ptr = ptr->pSeuraava;
98         }
99         ptr->pSeuraava = pUusi;
100        pUusi->pEdellinen = ptr;
101    }
102    return (pAlku);
103 }
```

4.7 paaohjelma.c File Reference

```
#include "binaaripuu.h"
#include "linkitettylista.h"
#include "yleiset.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

Functions

- int [main](#) (void)

Paaohjelma, pyorittaa koko ohjelmaa.

4.7.1 Function Documentation

4.7.1.1 main()

```
int main (
    void )
```

Returns

int Palauttaa aina 0, kun ohjelma tulee päätökseen.

Definition at line 13 of file paaohjelma.c.

```
13     {
14     int iValinta = 0;
15
16     do {
17         iValinta = valikko();
18         if (iValinta == 1) {
19             mainLista();
20         } else if (iValinta == 2) {
21             mainBinaaripuu();
22         } else if (iValinta == 0) {
23             printf("Lopetetaan.\n");
24         } else {
25             printf("Tuntematon valinta, yritä uudestaan.\n");
26         }
27         printf("\n");
28     } while (iValinta != 0);
29
30     printf("Kiitos ohjelman käytöstä.\n");
31     return (0);
32 }
```

4.8 punamustapuu.c File Reference

```
#include "binaaripuu.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

Functions

- [RBSOLMU * varaaMuistiaRB](#) (char *pNimi, int iArvo)
L10 / Punamustapuu Vaatii oman structin takia omat aliohjelmat, jotka ovat [binaaripuu.c](#) tiedoston aliohjelmiä muokattuina.
- [RBSOLMU * vapautaMuistiRB](#) (RBSOLMU *pJuuriSolmu)
- [RBSOLMU * lisaaRBSolmu](#) (RBSOLMU *pJuuriSolmu, RBSOLMU *pUusi)
- [RBSOLMU * luoRBPuu](#) (RBSOLMU *pJuuriSolmu, char *pNimi)
- void [kierraVasemmalle](#) (RBSOLMU **pJuuriSolmu, RBSOLMU *pSolmu)
- void [kierraOikealle](#) (RBSOLMU **pJuuriSolmu, RBSOLMU *pSolmu)
- void [korjaaLisays](#) (RBSOLMU **pJuuriSolmu, RBSOLMU *pUusi)
- void [kirjoitaRBTiedostoon](#) (char *pNimi, RBSOLMU *pJuuriSolmu)
- void [kirjoitaRB](#) (char *pNimi, RBSOLMU *pJuuriSolmu)

4.8.1 Function Documentation

4.8.1.1 kierraOikealle()

```
void kierraOikealle (
    RBSOLMU ** pJuurisolmu,
    RBSOLMU * pSolmu )
```

Definition at line 130 of file punamustapuu.c.

```
130 {
131     RBSOLMU *pVasenLapsi = pSolmu->pVasen;
132     pSolmu->pVasen = pVasenLapsi->pOikea;
133
134     if (pSolmu->pVasen != NULL)
135         pSolmu->pVasen->pVanhempi = pSolmu;
136
137     pVasenLapsi->pVanhempi = pSolmu->pVanhempi;
138
139     if (pSolmu->pVanhempi == NULL)
140         *pJuurisolmu = pVasenLapsi;
141     else if (pSolmu == pSolmu->pVanhempi->pVasen)
142         pSolmu->pVanhempi->pVasen = pVasenLapsi;
143     else
144         pSolmu->pVanhempi->pOikea = pVasenLapsi;
145
146     pVasenLapsi->pOikea = pSolmu;
147     pSolmu->pVanhempi = pVasenLapsi;
148 }
```

4.8.1.2 kierraVasemmalle()

```
void kierraVasemmalle (
    RBSOLMU ** pJuurisolmu,
    RBSOLMU * pSolmu )
```

Definition at line 109 of file punamustapuu.c.

```
109 {
110     RBSOLMU *pOikeaLapsi = pSolmu->pOikea;
111     pSolmu->pOikea = pOikeaLapsi->pVasen;
112
113     if (pSolmu->pOikea != NULL)
114         pSolmu->pOikea->pVanhempi = pSolmu;
115
116     // Solmun oikea lapsi nousee ylös puussa
117     pOikeaLapsi->pVanhempi = pSolmu->pVanhempi;
118
119     if (pSolmu->pVanhempi == NULL)
120         *pJuurisolmu = pOikeaLapsi;
121     else if (pSolmu == pSolmu->pVanhempi->pVasen)
122         pSolmu->pVanhempi->pVasen = pOikeaLapsi;
123     else
124         pSolmu->pVanhempi->pOikea = pOikeaLapsi;
125
126     pOikeaLapsi->pVasen = pSolmu;
127     pSolmu->pVanhempi = pOikeaLapsi;
128 }
```

4.8.1.3 kirjoitaRB()

```
void kirjoitaRB (
    char * pNimi,
    RBSOLMU * pJuurisolmu )
```

Definition at line 222 of file punamustapuu.c.

```
222                                     {
223     /*if (pJuurisolmu == NULL) {
224         printf("Puu on tyhjä, luo puurakenne ennen kirjoittamista.\n");
225         return;
226     }*/
227     if (pJuurisolmu != NULL) {
228         kirjoitaRBTiedostoon(pNimi, pJuurisolmu);
229         kirjoitaRB(pNimi, pJuurisolmu->pVasen);
230         kirjoitaRB(pNimi, pJuurisolmu->pOikea);
231     }
232     return;
233 }
```

4.8.1.4 kirjoitaRBTiedostoon()

```
void kirjoitaRBTiedostoon (
    char * pNimi,
    RBSOLMU * pJuurisolmu )
```

Definition at line 201 of file punamustapuu.c.

```
201                                     {
202     FILE *Tiedosto = NULL;
203
204     if (pJuurisolmu == NULL) {
205         return;
206     }
207
208     /* Tiedoston avaaminen. */
209     if ((Tiedosto = fopen(pNimi, "a")) == NULL) {
210         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
211         exit(0);
212     }
213
214     /* Tiedostoon kirjoittaminen. */
215     fprintf(Tiedosto, "%s,%d\n", pJuurisolmu->aNimi, pJuurisolmu->iArvo);
216
217     /* Tiedoston sulkeminen. */
218     fclose(Tiedosto);
219     printf("Tiedosto '%s' kirjoitettu.\n", pNimi);
220     return;
221 }
```

4.8.1.5 korjaaLisays()

```
void korjaaLisays (
    RBSOLMU ** pJuurisolmu,
    RBSOLMU * pUusi )
```

Definition at line 150 of file punamustapuu.c.

```
150                                     {
151     // Ilman p-alkua, koska pVanhempi->pVanhempi oli sekava
152     RBSOLMU *vanhempi = NULL;
153     RBSOLMU *seta = NULL;
154     RBSOLMU *isoVanhempi = NULL;
155
156     while ((pUusi != *pJuurisolmu) && (pUusi->pVanhempi->iVariBitti == 0)) {
157         vanhempi = pUusi->pVanhempi;
158         isoVanhempi = vanhempi->pVanhempi;
```

```

159
160     if (vanhemp1 == isoVanhemp1->pVasen) {
161         seta = isoVanhemp1->pOikea;
162         if (seta != NULL && seta->iVariBitti == 0) {
163             vanhemp1->iVariBitti = 1;
164             seta->iVariBitti = 1;
165             isoVanhemp1->iVariBitti = 0;
166             pUusi = isoVanhemp1;
167         } else {
168             if (pUusi == vanhemp1->pOikea) {
169                 pUusi = vanhemp1;
170                 kierraVasemmalle(pJuurisolmu, pUusi);
171                 vanhemp1 = pUusi->pVanhemp1;
172             }
173             vanhemp1->iVariBitti = 1;
174             isoVanhemp1->iVariBitti = 0;
175             kierraOikealle(pJuurisolmu, isoVanhemp1);
176         }
177     }
178     } else {
179         seta = isoVanhemp1->pVasen;
180         if (seta != NULL && seta->iVariBitti == 0) {
181             vanhemp1->iVariBitti = 1;
182             seta->iVariBitti = 1;
183             isoVanhemp1->iVariBitti = 0;
184             pUusi = isoVanhemp1;
185         } else {
186             if (pUusi == vanhemp1->pVasen) {
187                 pUusi = vanhemp1;
188                 kierraOikealle(pJuurisolmu, pUusi);
189                 vanhemp1 = pUusi->pVanhemp1;
190             }
191             vanhemp1->iVariBitti = 1;
192             isoVanhemp1->iVariBitti = 0;
193             kierraVasemmalle(pJuurisolmu, isoVanhemp1);
194         }
195     }
196 }
197 (*pJuurisolmu)->iVariBitti = 1;
198 }

```

4.8.1.6 lisaaRBSolmu()

```

RBSOLMU* lisaaRBSolmu (
    RBSOLMU * pJuurisolmu,
    RBSOLMU * pUusi )

```

Katsotaan, ovatko arvot samat. Laitetaan aakkosten perusteella vasemmalle tai oikealle.

Definition at line 38 of file punamustapuu.c.

```

38
39     int iVertailu = 0;
40
41     // Jos puu on tyhjä, palautetaan uusi RBSolmu.
42     if (pJuurisolmu == NULL) {
43         return pUusi;
44     }
45
46     iVertailu = strcmp(pUusi->aNimi, pJuurisolmu->aNimi);
47
48     // Solmujen lisääminen rekursiivisesti
49     if (pUusi->iArvo < pJuurisolmu->iArvo) {
50         pJuurisolmu->pVasen = lisaaRBSolmu(pJuurisolmu->pVasen, pUusi);
51         pJuurisolmu->pVasen->pVanhemp1 = pJuurisolmu;
52     } else if (pUusi->iArvo > pJuurisolmu->iArvo) {
53         pJuurisolmu->pOikea = lisaaRBSolmu(pJuurisolmu->pOikea, pUusi);
54         pJuurisolmu->pOikea->pVanhemp1 = pJuurisolmu;
55     } else if (pUusi->iArvo == pJuurisolmu->iArvo) {
56         if (iVertailu < 0) {
57             pJuurisolmu->pVasen = lisaaRBSolmu(pJuurisolmu->pVasen, pUusi);
58             pJuurisolmu->pVasen->pVanhemp1 = pJuurisolmu;
59         } else if (iVertailu > 0) {
60             pJuurisolmu->pOikea = lisaaRBSolmu(pJuurisolmu->pOikea, pUusi);
61             pJuurisolmu->pOikea->pVanhemp1 = pJuurisolmu;
62         }
63     }
64     }
65     return (pJuurisolmu);
66 }
67 }

```

4.8.1.7 luoRBPuu()

```
RBSOLMU* luoRBPuu (
    RBSOLMU * pJuurisolmu,
    char * pNimi )
```

Definition at line 69 of file punamustapuu.c.

```
69 {
70     FILE *Tiedosto = NULL;
71     char aRivi[LEN] = "";
72     int iOtsikko = 0;
73     char *p1 = NULL, *p2 = NULL;
74     int iArvo = 0;
75
76     if ((Tiedosto = fopen(pNimi, "r")) == NULL) {
77         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
78         exit(0);
79     }
80
81     while ((fgets(aRivi, LEN, Tiedosto)) != NULL) {
82         aRivi[strlen(aRivi) - 1] = '\0';
83         if (iOtsikko == 0) {
84             iOtsikko++;
85             continue;
86         }
87
88         if ((p1 = strtok(aRivi, ";")) == NULL) {
89             perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
90             exit(0);
91         }
92
93         if ((p2 = strtok(NULL, ";")) == NULL) {
94             perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
95             exit(0);
96         }
97
98         iArvo = atoi(p2);
99
100         RBSOLMU *pUusi = varaaMuistiaRB(p1, iArvo);
101         pJuurisolmu = lisaaRBSolmu(pJuurisolmu, pUusi);
102         korjaaLisays(&pJuurisolmu, pUusi);
103     }
104
105     fclose(Tiedosto);
106     return (pJuurisolmu);
107 }
```

4.8.1.8 vapautaMuistiRB()

```
RBSOLMU* vapautaMuistiRB (
    RBSOLMU * pJuuriSolmu )
```

Definition at line 28 of file punamustapuu.c.

```
28 {
29     if (pJuuriSolmu != NULL) {
30         vapautaMuistiRB(pJuuriSolmu->pVasen);
31         vapautaMuistiRB(pJuuriSolmu->pOikea);
32         free(pJuuriSolmu);
33     }
34     pJuuriSolmu = NULL;
35     return (pJuuriSolmu);
36 }
```

4.8.1.9 varaaMuistiaRB()

```
RBSOLMU* varaaMuistiaRB (
    char * pNimi,
    int iArvo )
```

Punamustapuu, ehkä oma header olisi kätevämpi, kun omat aliohjelmat oman structin takia.

Definition at line 11 of file punamustapuu.c.

```
11                                     {
12     RBSOLMU *pUusi = NULL;
13
14     if ((pUusi = (RBSOLMU *)malloc(sizeof(RBSOLMU))) == NULL) {
15         perror("Muistin varaus epäonnistui, lopetetaan");
16         exit(0);
17     }
18
19     pUusi->iVariBitti = 0; // uudet aina punaisia
20     strcpy(pUusi->aNimi, pNimi);
21     pUusi->iArvo = iArvo;
22     pUusi->pVasen = NULL;
23     pUusi->pOikea = NULL;
24     pUusi->pVanhempi = NULL;
25     return (pUusi);
26 }
```

4.9 yleiset.c File Reference

```
#include "yleiset.h"
#include "binaaripuu.h"
#include "haut.h"
#include "linkitettylista.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

Functions

- void [mainLista](#) (void)
Tekee käyttäjän valinnan perusteella listan toiminnot.
- void [mainBinaaripuu](#) (void)
Tekee käyttäjän valinnan perusteella binääripuun toiminnot.
- int [valikko](#) (void)
Ensimmäinen valikko, jossa käyttäjä valitsee listan tai binaaripuun.
- int [listaValikko](#) (void)
Tulostaa lista valikon ja kysyy käyttäjän valinnan.
- int [binaaripuuValikko](#) (void)
Tulostaa binaariluku valikon ja kysyy käyttäjältä valinnan.
- char * [kysyNimi](#) (char *pPrompti, char *pNimi)
Kysyy tiedoston/haettavan nimen.
- int [kysyArvo](#) (char *pPrompti, int iArvo)
Kysyy arvon, joka halutaan etsiä/poistaa puusta.

4.9.1 Function Documentation

4.9.1.1 binaaripuuValikko()

```
int binaaripuuValikko (
    void )
```

Parameters

<i>iValinta</i>	Kayttajan antama syote vaihtoehtojen pohjalta.
-----------------	--

Returns

int Palauttaa kayttajan valinnan.

Definition at line 240 of file yleiset.c.

```
240     {
241         int iValinta = 0;
242         printf("\nValitse haluamasi toiminto:\n");
243         printf("1) Luo puu\n");
244         printf("2) Kirjoita AVL puu tiedostoon\n");
245         printf("3) Syvyyshaku\n");
246         printf("4) Leveyshaku\n");
247         printf("5) Tyhjennä puu\n");
248         printf("6) Poista solmu\n");
249         printf("7) Binäärihaku AVL puusta\n");
250         printf("8) Punamustapuu\n");
251         printf("0) Takaisin\n");
252         printf("Anna valintasi: ");
253         scanf("%d", &iValinta);
254         getchar();
255         return iValinta;
256 }
```

4.9.1.2 kysyArvo()

```
int kysyArvo (
    char * pPrompti,
    int iArvo )
```

Parameters

<i>pPrompti</i>	Kysymys esim. "Anna haettava numeroarvo: "
<i>iArvo</i>	Arvo, joka halutaan poistaa/etsiä.

Returns

int Palauttaa kayttajan antaman arvon.

Definition at line 279 of file yleiset.c.

```
279     {
280         printf("%s", pPrompti);
281         scanf("%d", &iArvo);
282         getchar();
283         return (iArvo);
284 }
```

4.9.1.3 kysyNimi()

```
char* kysyNimi (
    char * pPrompti,
    char * pNimi )
```

Parameters

<i>pPrompti</i>	Kysymys esim. "Anna kirjoitettavan tiedoston nimi: "
<i>pNimi</i>	Tiedoston nimi/haettava nimi, jota halutaan kayttaa.

Returns

char Palauttaa kayttajan antaman merkkijonon.

Definition at line 265 of file yleiset.c.

```
265                                     {
266     printf("%s", pPrompti);
267     scanf("%s", pNimi);
268     getchar();
269     return (pNimi);
270 }
```

4.9.1.4 listaValikko()

```
int listaValikko (
    void )
```

Parameters

<i>iValinta</i>	Kayttajan antama syote vaihtoehtojen pohjalta.
-----------------	--

Returns

int Palauttaa kayttajan valinnan.

Definition at line 216 of file yleiset.c.

```
216                                     {
217     int iValinta = 0;
218     printf("\nValitse haluamasi toiminto:\n");
219     printf("1) Lue tiedosto\n");
220     printf("2) Kirjoita tiedosto alusta loppuun\n");
221     printf("3) Kirjoita tiedosto lopusta alkuun\n");
222     printf("4) Tyhjennä taulukko\n");
223     printf("5) Lajittele nousevasti\n");
224     printf("6) Lajittele laskevasti\n");
225     printf("7) Lisää tietoja listaan\n");
226     printf("8) Poista listalta tietoja\n");
227     printf("0) Takaisin\n");
228     printf("Anna valintasi: ");
229     scanf("%d", &iValinta);
230     getchar();
231     return iValinta;
232 }
```

4.9.1.5 mainBinaaripuu()

```
void mainBinaaripuu (
    void )
```

Funktio näyttää valikon ja suorittaa käyttäjän valinnan mukaiset binääripuuhun liittyvät toiminnot.

Definition at line 105 of file yleiset.c.

```
105     {
106     char aNimiLuettava[LEN] = "";
107     char aNimiKirjoitettava[LEN] = "";
108     char aHaettavaNimi[LEN] = "";
109     PUU *pJuuriSolmu = NULL;
110     RBSOLMU *pRBJuuri = NULL;
111     int iValinta = 0;
112     int iArvo = 0;
113
114     do {
115         iValinta = binaaripuuValikko();
116         if (iValinta == 1) {
117             kysyNimi("Tiedoston nimi: ", aNimiLuettava);
118             pJuuriSolmu = luoPuu(aNimiLuettava, pJuuriSolmu);
119
120         } else if (iValinta == 2) {
121             if (pJuuriSolmu == NULL) {
122                 printf("Luo binääripuu ennen sen kirjoittamista.\n");
123             } else {
124                 kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
125                 kirjoitaBinaaripuu(aNimiKirjoitettava, pJuuriSolmu);
126             }
127
128         } else if (iValinta == 3) {
129             if (pJuuriSolmu == NULL) {
130                 printf("Luo binääripuu ennen syvyyshakua.\n");
131             } else {
132                 iArvo = kysyArvo("Anna haettava numeroarvo: ", iArvo);
133                 kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
134                 tarkistaLoytyykoSyvyyshaulla(aNimiKirjoitettava, pJuuriSolmu, iArvo);
135             }
136
137         } else if (iValinta == 4) {
138             if (pJuuriSolmu == NULL) {
139                 printf("Luo binääripuu ennen leveyshakua.\n");
140             } else {
141                 kysyNimi("Anna haettava nimi: ", aHaettavaNimi);
142                 kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
143                 leveyshaku(aNimiKirjoitettava, pJuuriSolmu, aHaettavaNimi);
144             }
145
146         } else if (iValinta == 5) {
147             pJuuriSolmu = vapautaMuistiPuu(pJuuriSolmu);
148             printf("Muisti vapautettu.\n");
149
150         } else if (iValinta == 6) {
151             if (pJuuriSolmu == NULL) {
152                 printf("Luo binääripuu ennen poistoa.\n");
153             } else {
154                 kysyNimi("Anna poistettava nimi tai arvo: ", aHaettavaNimi);
155                 pJuuriSolmu = poistaSolmu(aHaettavaNimi, pJuuriSolmu);
156             }
157
158         } else if (iValinta == 7) {
159             if (pJuuriSolmu == NULL) {
160                 printf("Luo binääripuu ennen binäärihakua.\n");
161             } else {
162                 kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
163                 iArvo = kysyArvo("Anna haettava numeroarvo: ", iArvo);
164                 iArvo = binaarihakua(aNimiKirjoitettava, pJuuriSolmu, iArvo);
165                 if (iArvo == 0) {
166                     printf("Hakemaasi arvoa ei löytynyt binääripuusta.\n");
167                 }
168             }
169
170         } else if (iValinta == 8) {
171             kysyNimi("Anna luettavan tiedoston nimi: ", aNimiLuettava);
172             kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
173             pRBJuuri = luoRBPuu(pRBJuuri, aNimiLuettava);
174             kirjoitaRB(aNimiKirjoitettava, pRBJuuri);
175             pRBJuuri = vapautaMuistiRB(pRBJuuri);
176
177         } else if (iValinta == 0) {
178             printf("Palataan takaisin päävalikkoon.\n");
179         }
```

```

180         } else {
181             printf("Tuntematon valinta, yritä uudestaan.\n");
182         }
183     } while (iValinta != 0);
184
185     /* Vapautetaan puun varaama muisti varmuuden vuoksi vielä täällä lopussa erikseen, jos
186        * käyttäjä ei muista sitä tehdä. */
187
188     pJuuriSolmu = vapautaMuistiPuu(pJuuriSolmu);
189     return;
190 }

```

4.9.1.6 mainLista()

```

void mainLista (
    void )

```

Funktio näyttää valikon ja suorittaa käyttäjän valinnan mukaiset listaan liittyvät toiminnot.

Definition at line 17 of file yleiset.c.

```

17     {
18         char aNimiLuettava[LEN] = "";
19         char aNimiKirjoitettava[LEN] = "";
20         char aTietoPoistettava[LEN] = "";
21         int iArvo = 0;
22         int iIndeksi = 0;
23         int iValinta = 0;
24         //int iSamaLKM = 0;
25         TIEDOT *pAlku = NULL;
26
27         do {
28             iValinta = listaValikko();
29             if (iValinta == 1) {
30                 kysyNimi("Anna luettavan tiedoston nimi: ", aNimiLuettava);
31                 pAlku = lue(aNimiLuettava, pAlku);
32             }
33             else if (iValinta == 2) {
34                 if (pAlku == NULL) {
35                     printf("Lue tiedosto ennen kirjoittamista.\n");
36                 } else {
37                     kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
38                     kirjoitaTiedostoAlustaloppuun(aNimiKirjoitettava, pAlku);
39                 }
40             }
41             else if (iValinta == 3) {
42                 if (pAlku == NULL) {
43                     printf("Lue tiedosto ennen kirjoittamista.\n");
44                 } else {
45                     kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
46                     kirjoitaTiedostoLopustaAlkuun(aNimiKirjoitettava, pAlku);
47                 }
48             }
49             else if (iValinta == 4) {
50                 pAlku = vapautaMuisti(pAlku);
51                 printf("Muisti vapautettu.\n");
52             }
53             else if (iValinta == 5) {
54                 pAlku = lomitustaLajittelu(pAlku);
55                 printf("Numerot lajiteltu nousevasti.\n");
56             }
57             else if (iValinta == 6) {
58                 pAlku = lisaysLajittelu(pAlku);
59                 printf("Numerot lajiteltu laskevasti.\n");
60             }
61             else if (iValinta == 7) {
62                 iIndeksi = kysyArvo("Mille riville haluat lisätä tietoja: ", iIndeksi);
63                 kysyNimi("Anna lisättävä nimi: ", aNimiKirjoitettava);
64                 iArvo = kysyArvo("Anna tämän nimisten lukumäärä: ", iArvo);
65                 pAlku = lisaaListaan(pAlku, iIndeksi, aNimiKirjoitettava, iArvo);
66             }
67             else if (iValinta == 8) {
68                 // Poistaminen listalta
69
70                 /*iArvo = kysyArvo("Anna poistettava lukumäärä: ", iArvo);
71                 iSamaLKM = useammallaAlkiollaSamaLKM(pAlku, iArvo);
72                 if (iSamaLKM == 1) {
73                     pAlku = poistaListastaLkmPeruusteella(pAlku, iArvo);

```

```

74         } else if (iSamaLKM > 1) {
75             kysyNimi("Anna poistettava nimi: ", aNimiKirjoitettava);
76             pAlku = poistaListastaNimenPerusteella(pAlku, aNimiKirjoitettava);
77         }*/
78
79         kysyNimi("Anna poistettava tieto: ", aTietoPoistettava);
80         iArvo = onkoLukuVaiNimi(aTietoPoistettava);
81         pAlku = poistaListastaAlkio(pAlku, iArvo, aTietoPoistettava);
82         //TIEDOT *poistaListastaAlkio(TIEDOT *pAlku, int iLuvuVaiNimi, char *pTieto)
83
84     } else if (iValinta == 0) {
85         printf("Palataan takaisin päävalikkoon.\n");
86
87     } else {
88         printf("Tuntematon valinta, yritä uudestaan.\n");
89     }
90 } while (iValinta != 0);
91
92 /* Vapautetaan listan varaama muisti varmuuden vuoksi vielä täällä lopussa erikseen, jos
93  * käyttäjä ei muista sitä tehdä. */
94
95 pAlku = vapautaMuisti(pAlku);
96 return;
97 }

```

4.9.1.7 valikko()

```

int valikko (
    void )

```

Parameters

<i>iValinta</i>	Kayttajan antama syote.
-----------------	-------------------------

Returns

int Palauttaa kayttajan valinnan.

Definition at line 198 of file yleiset.c.

```

198     {
199         int iValinta = 0;
200         printf("Valitse haluamasi toiminto:\n");
201         printf("1) Lista\n");
202         printf("2) Binääripuu\n");
203         printf("0) Lopeta\n");
204         printf("Anna valintasi: ");
205         scanf("%d", &iValinta);
206         getchar();
207         return iValinta;
208     }

```

4.10 yleiset.h File Reference

Macros

- #define `LEN` 50

Functions

- void `mainLista` (void)
Tekee käyttäjän valinnan perusteella listan toiminnot.
- void `mainBinaaripuu` (void)
Tekee käyttäjän valinnan perusteella binääripuun toiminnot.
- int `valikko` (void)
Ensimmäinen valikko, jossa käyttäjä valitsee listan tai binaaripuun.
- int `listaValikko` (void)
Tulostaa lista valikon ja kysyy käyttäjän valinnan.
- int `binaaripuuValikko` (void)
Tulostaa binaariluku valikon ja kysyy käyttäjältä valinnan.
- char * `kysyNimi` (char *pPrompti, char *pNimi)
Kysyy tiedoston/haettavan nimen.
- int `kysyArvo` (char *pPrompti, int iArvo)
Kysyy arvon, joka halutaan etsiä/poistaa puusta.

4.10.1 Macro Definition Documentation

4.10.1.1 LEN

```
#define LEN 50
```

Definition at line 4 of file yleiset.h.

4.10.2 Function Documentation

4.10.2.1 binaaripuuValikko()

```
int binaaripuuValikko (
    void )
```

Parameters

<i>iValinta</i>	Käyttäjän antama syöte vaihtoehtojen pohjalta.
-----------------	--

Returns

int Palauttaa käyttäjän valinnan.

Definition at line 240 of file yleiset.c.

```
240 {
```

```

241     int iValinta = 0;
242     printf("\nValitse haluamasi toiminto:\n");
243     printf("1) Luo puu\n");
244     printf("2) Kirjoita AVL puu tiedostoon\n");
245     printf("3) Syvyyshaku\n");
246     printf("4) Leveyshaku\n");
247     printf("5) Tyhjennä puu\n");
248     printf("6) Poista solmu\n");
249     printf("7) Binäärihaku AVL puusta\n");
250     printf("8) Punamustapuu\n");
251     printf("0) Takaisin\n");
252     printf("Anna valintasi: ");
253     scanf("%d", &iValinta);
254     getchar();
255     return iValinta;
256 }

```

4.10.2.2 kysyArvo()

```

int kysyArvo (
    char * pPrompti,
    int iArvo )

```

Parameters

<i>pPrompti</i>	Kysymys esim. "Anna haettava numeroarvo: "
<i>iArvo</i>	Arvo, joka halutaan poistaa/etsiä.

Returns

int Palauttaa käyttäjän antaman arvon.

Definition at line 279 of file yleiset.c.

```

279                                     {
280     printf("%s", pPrompti);
281     scanf("%d", &iArvo);
282     getchar();
283     return (iArvo);
284 }

```

4.10.2.3 kysyNimi()

```

char* kysyNimi (
    char * pPrompti,
    char * pNimi )

```

Parameters

<i>pPrompti</i>	Kysymys esim. "Anna kirjoitettavan tiedoston nimi: "
<i>pNimi</i>	Tiedoston nimi/haettava nimi, jota halutaan kayttää.

Returns

char Palauttaa kayttajan antaman merkkijonon.

Definition at line 265 of file yleiset.c.

```

265                                     {
266     printf("%s", pPrompti);
267     scanf("%s", pNimi);
268     getchar();
269     return (pNimi);
270 }
```

4.10.2.4 listaValikko()

```

int listaValikko (
    void )
```

Parameters

<i>iValinta</i>	Kayttajan antama syote vaihtoehtojen pohjalta.
-----------------	--

Returns

int Palauttaa kayttajan valinnan.

Definition at line 216 of file yleiset.c.

```

216     {
217     int iValinta = 0;
218     printf("\nValitse haluamasi toiminto:\n");
219     printf("1) Lue tiedosto\n");
220     printf("2) Kirjoita tiedosto alusta loppuun\n");
221     printf("3) Kirjoita tiedosto lopusta alkuun\n");
222     printf("4) Tyhjennä taulukko\n");
223     printf("5) Lajittele nousevasti\n");
224     printf("6) Lajittele laskevasti\n");
225     printf("7) Lisää tietoja listaan\n");
226     printf("8) Poista listalta tietoja\n");
227     printf("0) Takaisin\n");
228     printf("Anna valintasi: ");
229     scanf("%d", &iValinta);
230     getchar();
231     return iValinta;
232 }
```

4.10.2.5 mainBinaaripuu()

```

void mainBinaaripuu (
    void )
```

Funktio näyttää valikon ja suorittaa käyttäjän valinnan mukaiset binääripuuhun liittyvät toiminnot.

Definition at line 105 of file yleiset.c.

```

105     {
106     char aNimiLuettava[LEN] = "";
107     char aNimiKirjoitettava[LEN] = "";
108     char aHaettavaNimi[LEN] = "";
109     PUU *pJuuriSolmu = NULL;
110     RBSOLMU *pRBJuuri = NULL;
111     int iValinta = 0;
```



```

112     int iArvo = 0;
113
114     do {
115         iValinta = binaaripuuValikko();
116         if (iValinta == 1) {
117             kysyNimi("Tiedoston nimi: ", aNimiLuettava);
118             pJuuriSolmu = luoPuu(aNimiLuettava, pJuuriSolmu);
119
120         } else if (iValinta == 2) {
121             if (pJuuriSolmu == NULL) {
122                 printf("Luo binääripuu ennen sen kirjoittamista.\n");
123             } else {
124                 kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
125                 kirjoitaBinaaripuu(aNimiKirjoitettava, pJuuriSolmu);
126             }
127
128         } else if (iValinta == 3) {
129             if (pJuuriSolmu == NULL) {
130                 printf("Luo binääripuu ennen syvyyshakua.\n");
131             } else {
132                 iArvo = kysyArvo("Anna haettava numeroarvo: ", iArvo);
133                 kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
134                 tarkistaLoytyykoSyvyysshaula(aNimiKirjoitettava, pJuuriSolmu, iArvo);
135             }
136
137         } else if (iValinta == 4) {
138             if (pJuuriSolmu == NULL) {
139                 printf("Luo binääripuu ennen leveyshakua.\n");
140             } else {
141                 kysyNimi("Anna haettava nimi: ", aHaettavaNimi);
142                 kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
143                 leveyshaku(aNimiKirjoitettava, pJuuriSolmu, aHaettavaNimi);
144             }
145
146         } else if (iValinta == 5) {
147             pJuuriSolmu = vapautaMuistiPuu(pJuuriSolmu);
148             printf("Muisti vapautettu.\n");
149
150         } else if (iValinta == 6) {
151             if (pJuuriSolmu == NULL) {
152                 printf("Luo binääripuu ennen poistoa.\n");
153             } else {
154                 kysyNimi("Anna poistettava nimi tai arvo: ", aHaettavaNimi);
155                 pJuuriSolmu = poistaSolmu(aHaettavaNimi, pJuuriSolmu);
156             }
157
158         } else if (iValinta == 7) {
159             if (pJuuriSolmu == NULL) {
160                 printf("Luo binääripuu ennen binäärihakua.\n");
161             } else {
162                 kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
163                 iArvo = kysyArvo("Anna haettava numeroarvo: ", iArvo);
164                 iArvo = binaarihaku(aNimiKirjoitettava, pJuuriSolmu, iArvo);
165                 if (iArvo == 0) {
166                     printf("Hakemaasi arvoa ei löytynyt binääripuusta.\n");
167                 }
168             }
169
170         } else if (iValinta == 8) {
171             kysyNimi("Anna luettavan tiedoston nimi: ", aNimiLuettava);
172             kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
173             pRBJuuri = luoRBPuu(pRBJuuri, aNimiLuettava);
174             kirjoitaRB(aNimiKirjoitettava, pRBJuuri);
175             pRBJuuri = vapautaMuistiRB(pRBJuuri);
176
177         } else if (iValinta == 0) {
178             printf("Palataan takaisin päävalikkoon.\n");
179
180         } else {
181             printf("Tuntematon valinta, yritä uudestaan.\n");
182         }
183     } while (iValinta != 0);
184
185     /* Vapautetaan puun varaama muisti varmuuden vuoksi vielä täällä lopussa erikseen, jos
186     * käyttäjä ei muista sitä tehdä. */
187
188     pJuuriSolmu = vapautaMuistiPuu(pJuuriSolmu);
189     return;
190 }

```

4.10.2.6 mainLista()

```
void mainLista (
    void )
```

Funktio näyttää valikon ja suorittaa käyttäjän valinnan mukaiset listaan liittyvät toiminnot.

Definition at line 17 of file yleiset.c.

```
17     {
18     char aNimiLuettava[LEN] = "";
19     char aNimiKirjoitettava[LEN] = "";
20     char aTietoPoistettava[LEN] = "";
21     int iArvo = 0;
22     int iIndeksi = 0;
23     int iValinta = 0;
24     //int iSamaLKM = 0;
25     TIEDOT *pAlku = NULL;
26
27     do {
28         iValinta = listaValikko();
29         if (iValinta == 1) {
30             kysyNimi("Anna luettavan tiedoston nimi: ", aNimiLuettava);
31             pAlku = lue(aNimiLuettava, pAlku);
32
33         } else if (iValinta == 2) {
34             if (pAlku == NULL) {
35                 printf("Lue tiedosto ennen kirjoittamista.\n");
36             } else {
37                 kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
38                 kirjoitaTiedostoAlustaLoppuun(aNimiKirjoitettava, pAlku);
39             }
40
41         } else if (iValinta == 3) {
42             if (pAlku == NULL) {
43                 printf("Lue tiedosto ennen kirjoittamista.\n");
44             } else {
45                 kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
46                 kirjoitaTiedostoLopustaAlkuun(aNimiKirjoitettava, pAlku);
47             }
48
49         } else if (iValinta == 4) {
50             pAlku = vapautaMuisti(pAlku);
51             printf("Muisti vapautettu.\n");
52
53         } else if (iValinta == 5) {
54             pAlku = lomitusLajittelu(pAlku);
55             printf("Numerot lajiteltu nousevasti.\n");
56
57         } else if (iValinta == 6) {
58             pAlku = lisaysLajittelu(pAlku);
59             printf("Numerot lajiteltu laskevasti.\n");
60
61         } else if (iValinta == 7) {
62             iIndeksi = kysyArvo("Mille riville haluat lisätä tietoja: ", iIndeksi);
63             kysyNimi("Anna lisättävä nimi: ", aNimiKirjoitettava);
64             iArvo = kysyArvo("Anna tämän nimisten lukumäärä: ", iArvo);
65             pAlku = lisaaListaan(pAlku, iIndeksi, aNimiKirjoitettava, iArvo);
66
67         } else if (iValinta == 8) {
68             // Poistaminen listalta
69
70             /*iArvo = kysyArvo("Anna poistettava lukumäärä: ", iArvo);
71             iSamaLKM = useammallaAlkiollaSamaLKM(pAlku, iArvo);
72             if (iSamaLKM == 1) {
73                 pAlku = poistaListastaLkmPeruusteella(pAlku, iArvo);
74             } else if (iSamaLKM > 1) {
75                 kysyNimi("Anna poistettava nimi: ", aNimiKirjoitettava);
76                 pAlku = poistaListastaNimenPerusteella(pAlku, aNimiKirjoitettava);
77             }*/
78
79             kysyNimi("Anna poistettava tieto: ", aTietoPoistettava);
80             iArvo = onkoLukuVaiNimi(aTietoPoistettava);
81             pAlku = poistaListastaAlkio(pAlku, iArvo, aTietoPoistettava);
82             //TIEDOT *poistaListastaAlkio(TIEDOT *pAlku, int iLuvuVaiNimi, char *pTieto)
83
84         } else if (iValinta == 0) {
85             printf("Palataan takaisin päävalikkoon.\n");
86
87         } else {
88             printf("Tuntematon valinta, yritä uudestaan.\n");
89         }
90     } while (iValinta != 0);
91 }
```

```
92     /* Vapautetaan listan varaama muisti varmuuden vuoksi vielä täällä lopussa erikseen, jos
93     * käyttäjä ei muista sitä tehdä. */
94
95     pAlku = vapautaMuisti(pAlku);
96     return;
97 }
```

4.10.2.7 valikko()

```
int valikko (
    void )
```

Parameters

<i>iValinta</i>	Kayttajan antama syote.
-----------------	-------------------------

Returns

int Palauttaa kayttajan valinnan.

Definition at line 198 of file yleiset.c.

```
198     {
199         int iValinta = 0;
200         printf("Valitse haluamasi toiminto:\n");
201         printf("1) Lista\n");
202         printf("2) Binääripuu\n");
203         printf("0) Lopeta\n");
204         printf("Anna valintasi: ");
205         scanf("%d", &iValinta);
206         getchar();
207         return iValinta;
208     }
```

